



ANALYSE SYNTHÈSE DEPARTMENT

# Toward Controllable Music Editing tools:

*Non Causal Attention and Musical Control in Autoregressive Music Generation Models*

---

Master's Thesis

Written by

**Emilio Picard**

**IRCAM – STMS Lab**

Sorbonne Université & Télécom Paris

Paris, France

**Supervision**

**Primary Supervisor:**  
Dr. Doras Guillaume

**Co-Supervisors:**  
Dr. Roebel Axel  
Amadu Frederic

**Date:**  
August 20, 2026

# Abstract

Autoregressive music generation models produce convincing clips but offer composers no way to revise one: a causal decoder can only continue a prompt, never regenerate an interior window while honoring what comes after it, and every such model is bounded to a context of ten to thirty seconds, far short of a finished piece. This report casts editing as infilling — sampling an interior window conditioned on both its prefix and its suffix — and reaches it with a single architectural change: an asymmetric span attention mask that lets the edit window attend to the entire context, past and future, while every other position keeps its ordinary causal view. Combined with an edit-mode training recipe, this turns a causal MusicGen-style model into a native infiller without touching sampling, guidance, or decoding, and reduces exactly to the original causal model as a special case. On top of this mechanism we build a composer’s control layer: a cosine context schedule dialing how strongly the edit adheres to each side of its context, melody conditioning via a chromagram of the edit window, and a dataset-token control routed through adaptive layer normalization. The same mask, composed with a stream selection over MusicGen-Stem’s per-instrument codebooks, further yields a stem-by-time editor that regenerates one chosen instrument inside one chosen window, and chaining that editor’s own continuation mode with its own infilling pass reaches compositions longer than any single context window — this internship’s culminating goal. Every model is trained from scratch on two 48 GB GPUs, on small, deliberately mismatched corpora, and evaluated against causal and public-checkpoint baselines; a web demo at <https://emilio-pcrd.github.io/editgen-demo/> lets users listen to generated samples, and a local prototype, *Stem Studio*, puts the stem-by-time editor behind a DAW-like timeline for interactive, track-by-track editing. Results confirm each mechanism behaves as the method predicts, while underscoring how much remains provisional at this scale and sample size — a deliberate trade-off for a technique meant to remain reachable outside an industrial compute budget. The code is available at <https://github.com/emilio-pcrd/music-stem-edit>.

# Acknowledgements

I would like to thank my supervisors, Guillaume Doras, Axel Roebel, and Frederic Amadu, for their guidance and support during this internship. Huge thanks to Mehdi, Ivan and all the other interns and visiting students of the lab for their really interesting talks and laughs throughout the internship.

## Compute and AI Transparency Statement

**Training Compute:** The experimental phase involved training  $\sim 300$ M parameter autoregressive (MusicGen-style) music generation models across multiple training runs, totaling approximately 720 hours of computation. Experiments were conducted on a local academic cluster using two NVIDIA RTX A6000 GPUs (48GB VRAM each, running at  $\sim 45$ GB capacity in parallel). Based on a thermal design power of 300W per GPU, the cumulative direct energy consumption for model training is estimated at approximately 432 kWh.

**AI Assistance:** In accordance with emerging academic transparency guidelines, I acknowledge the use of Anthropic’s Claude (Opus) via Claude Code during this internship. It was used as an assistant for code refactoring, manuscript proofreading, and the rapid prototyping of a local DAW-integration interface. All generated suggestions, implementations, and text were critically reviewed and modified by myself, who assumes full responsibility for the final work.

# Contents

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Abstract</b>                                       | <b>1</b>  |
| <b>Acknowledgements</b>                               | <b>2</b>  |
| <b>1 Introduction</b>                                 | <b>4</b>  |
| 1.1 Context and motivation . . . . .                  | 4         |
| 1.2 Edition as infilling . . . . .                    | 5         |
| 1.3 From one window to a whole piece . . . . .        | 6         |
| 1.4 Contributions and outline . . . . .               | 6         |
| <b>2 Theoretical Foundations</b>                      | <b>8</b>  |
| 2.1 Neural audio codecs . . . . .                     | 8         |
| 2.2 Autoregressive generation: MusicGen . . . . .     | 10        |
| 2.3 Editing & infilling: state of the art . . . . .   | 13        |
| 2.4 Controllability . . . . .                         | 15        |
| 2.5 Multi-stem generation and MusicGen-Stem . . . . . | 17        |
| <b>3 Method</b>                                       | <b>19</b> |
| 3.1 Editing as infilling . . . . .                    | 19        |
| 3.2 Controlling generation . . . . .                  | 26        |
| 3.3 Stem-by-time editing . . . . .                    | 33        |
| <b>4 Experiments &amp; Results</b>                    | <b>37</b> |
| 4.1 Setup . . . . .                                   | 37        |
| 4.2 Text-to-music generation . . . . .                | 39        |
| 4.3 Conditioning and control . . . . .                | 41        |
| 4.4 Editing . . . . .                                 | 42        |
| <b>5 Discussion and Conclusion</b>                    | <b>45</b> |
| 5.1 Contributions and evidence . . . . .              | 45        |
| 5.2 Discussion . . . . .                              | 46        |
| 5.3 Future work and perspectives . . . . .            | 47        |

# Chapter 1

## Introduction

### 1.1 Context and motivation

Consider a composer at work. A piece rarely arrives whole: it is sketched, revised, part by part; a passage is kept, another discarded, a transition reworked while everything around it stays in place. Digital audio workstations institutionalized this way of working through non-destructive editing on a timeline — a change is always local, always reversible, always auditioned against what surrounds it. Any tool that hopes to find a place in the studio inherits these expectations: it must act on *this* passage, follow the composer’s musical intentions — a melody to respect, a sound world to draw from — and invite retry after retry until the result sits well.

The starting question of this report is therefore not whether machines can compose in our place, but whether they can be composed *with*, the position Tokui [31] defends under the title *AI for creating*: systems that expand what an artist can do, the way an instrument does, rather than automate the artist away. Language models trained on discrete audio tokens — Jukebox [11], AudioLM [7], MusicLM [2], MusicGen [9] — and latent diffusion models [13] now produce full-band musical clips that are locally coherent and stylistically convincing, resting on neural audio codecs [10, 36] that compress waveforms into token sequences a Transformer can model. Measured against the composer’s workflow, though, these models remain one-shot instruments: a prompt goes in, a clip comes out, and the exchange ends there. A composer who likes everything in a clip *except* one bar can only resample the whole clip, losing exactly the material they meant to keep, and a text prompt cannot say *which* melody the redone bar should follow. Becoming an instrument rather than a slot machine requires the two things the studio workflow takes for granted: *local regeneration* — redo this window, keep the rest — and *musical control* over what fills it.

This report investigates both questions for autoregressive (AR) music language models of the MusicGen family, in a deliberately modest computational regime: every

model here trains *from scratch* on two 48 GB GPUs, on music corpora (a pop-jazz-country music corpus) and SCHAEFFER [6], a small database of human-annotated electroacoustic sound objects — rather than fine-tuned from a large pretrained checkpoint. This constraint reflects the resources actually available to academic labs and closer to independent artists, and shapes every design choice toward modifications of the training recipe rather than new architectures or inference machinery.

Local regeneration and musical control turn out to compose cleanly with a third question: a composer’s workstation is several *tracks*, not one waveform, and a real piece runs minutes, far longer than any context window that a language model is able to go at one time.

## 1.2 Edition as infilling

We see editing as *infilling*. A clip is partitioned into a prefix  $P$ , an edit window  $E$ , and a suffix  $S$ ; editing means sampling  $E$  from  $p(E \mid P, S)$  so it connects with both its past and its future. A causal AR model factorizes strictly left-to-right, so no token in  $E$  ever sees  $S$ : it can *continue* from the start of  $E$  onward, but the suffix is then discarded rather than respected.

The area of research has responded by leaving the AR paradigm: masked iterative decoders in the MaskGIT lineage [8], such as VampNet [15] and MAGNeT [37], resample any masked subset of tokens conditioned on the rest, at the cost of the AR factorization, single-pass decoding, and exact KV caching. Text and code modeling instead keep the causal decoder and rearrange the sequence so the suffix precedes the middle [4, 12, 14], also proposed in the music MIDI domain in Piano Inpainting Application for symbolic music [18].

Our approach keeps the AR decoder and its left-to-right generation, but change *what the edit window is allowed to see*. An *asymmetric span mask* lets context tokens ( $P, S$ ) attend causally among themselves, exactly as in the base model, while tokens in  $E$  additionally attend to the entire context, past *and* future, still strictly left-to-right inside the window, so sampling and classifier-free guidance [19] carry over unchanged. Turning a generator into an editor then reduces to a training-recipe change rather than a new architecture. On top of it we build the control layer composers need: an inference-time schedule trading fidelity to context against freedom inside the window, melody conditioning via chromagrams, and a dataset token, injected through adaptive layer normalization (AdaLN) [24], steering the edit toward the pop corpus or the Schaeffer sound world.

### 1.3 From one window to a whole piece

Everything above edits a *mix*. Keep the drums and vocals, redo the bassline in this bar, is at least as common a request as “redo everything in this bar,” and nothing so far can honor it: the mask controls *when*  $E$  starts and ends, not *which instrument* inside it gets touched. A second, more basic limitation is context length itself: every model here, like nearly every AR music language model, generates within ten to thirty seconds, far short of an actual two-to-four-minute song. A tool that can revise a fragment cannot yet be asked to work on the piece.

This report closes both gaps with machinery already on the table, composed rather than reinvented. MusicGen-Stem [29] tokenizes a clip as several parallel, independently-quantized code streams, one group per stem (drums, bass, and the rest, separated with Hybrid Transformer Demucs [27]), read by one joint language model. Because the asymmetric mask acts on *attention*, and because *which streams count as already generated* is already a free choice, the two mechanisms compose with no new masking machinery:  $M^*$  supplies the *when*, a stream selection over MusicGen-Stem’s codebooks the *which instrument*, together a stem-by-time editor (Section 3.3.1) — the point this internship’s work reaches. The second gap is left open. Because that editor is still an ordinary AR decoder, its own **continuation** mode can seed one bounded window after another toward an arbitrary length, and stitching each new seam with an **interior** edit is a schedule over primitives this report already provides; but chaining was not evaluated here, and Section 5.3 argues that a model trained with sliding-window attention is the better answer to a whole piece.

### 1.4 Contributions and outline

The contributions of this internship are:

- (i) an **asymmetric span attention mask** and **edit-mode training recipe** turning a causal MusicGen-style model into an editor sampling from  $p(E \mid P, S)$ , preserving plain-generation behavior and exact KV caching (Sections 3.1.2–3.1.3);
- (ii) a **cosine context schedule** dialing how strongly the edit adheres to each side of its context (Section 3.2.1);
- (iii) **melody-conditioned editing** via an edit-window chromagram (Section 3.2.2);
- (iv) **dataset-token control via AdaLN**, a binary flag selecting the pop corpus or the Schaeffer sound world at edit time (Section 3.2.3);
- (v) a **stem-by-time editor** composing the mask with per-instrument streams from MusicGen-Stem, plus the base multi-stem generator: codebook budgets, a coarse-to-fine regeneration recipe, chroma conditioning, and a bass loss weight (Sec-

tion 3.3.1);

All models are trained from scratch on  $2 \times 48$  GB GPUs, demonstrating that AR music editing — time-local, stem-local, and eventually piece-length — is reachable in a small-compute academic regime. Alongside these mechanisms, a local prototype, *Stem Studio* (Section 5.3), puts the stem-by-time editor behind a DAW-like timeline — four stem tracks, drag-to-select a window, generate or infill it — the composer-facing form this introduction’s opening scene asked for.

Chapter 2 reviews neural audio codecs, MusicGen, the state of the art in editing and control, and multi-stem generation. Chapter 3 presents the method — editing, control, and the stem-by-time editor. Chapter 4 evaluates each component; Chapter 5 discusses the results, the comparison with non-AR editors, and the limitations of the small-compute regime; Chapter 5 concludes with perspectives toward composer-facing tools that work across a whole piece and every one of its tracks.

# Chapter 2

## Theoretical Foundations

In this chapter, we recall basic elements of autoregressive music generation, the task of producing a sequence of discrete symbols representing a musical waveform. Four questions organize it: how a continuous pressure wave becomes a sequence of discrete symbols a language model can read and write (Section 2.1); what it means, mathematically, to *generate* music, how such a model is trained and sampled (Section 2.2), and what the state of the art offers for *editing* — regenerating a window inside an existing piece (Section 2.3); and through which doors musical intentions enter the network (Section 2.4).

**Notation.** We use the following notation consistently throughout:

$x$  a waveform.

$z$  a continuous latent.

$u, U$  a discrete token, and a sequence of discrete tokens.

$p_\theta$  a model with parameters  $\theta$ , approximating the true distribution  $p^\star$ .

$c$  conditioning information.

$M$  an attention mask.

$P, E, S$  the prefix–edit–suffix partition, as introduced in Chapter 1.

---

### 2.1 Neural audio codecs

Digital audio begins with sampling: a microphone signal measured  $f_s$  times per second gives  $x = (x_1, \dots, x_N) \in [-1, 1]^N$ ,  $N = f_s \cdot \tau$  for a  $\tau$ -second recording. At  $f_s = 32$  kHz, the rate MusicGen uses, a 30-second clip is 960,000 numbers. Modeling music directly at this resolution is possible — WaveNet [32] did so, autoregressively, over raw samples — but the sequence lengths make training and sampling costly.

One solution is to *downsample* the sample rate; neural audio codecs [10, 36] learn a compressor whose codewords play a vocabulary’s role: one second of music becomes a few symbols from a learned alphabet rather than 32,000 reals. Formally, it is a triple  $(\mathcal{E}, Q, \mathcal{D})$ : an encoder mapping the waveform to a downsampled sequence of latents, a quantizer replacing each latent with a codebook symbol, and a decoder mapping symbols back to a waveform,

$$x \xrightarrow{\mathcal{E}} (z_1, \dots, z_T), \quad z_t \in \mathbb{R}^d \xrightarrow{Q} (u_1, \dots, u_T) \xrightarrow{\mathcal{D}} \hat{x} \approx x. \quad (2.1)$$

The encoder’s overall temporal stride  $M$  sets the latent *frame rate*  $f_r = f_s/M$ ; for the EnCodec model used throughout,  $M = 640$  and  $f_r = 50$  Hz.

### 2.1.1 Quantization: VQ, RVQ, and EnCodec

From auto-encoder architectures, the encoder output is usually continuous, but an alphabet must be finite. The classical remedy is *vector quantization* (VQ) [17, 33]: the idea is to learn a codebook  $\mathcal{C} = \{e_1, \dots, e_V\} \subset \mathbb{R}^d$  and replace each latent by its nearest prototype’s index,

$$Q_{\mathcal{C}}(z) = \arg \min_{v \in \{1, \dots, V\}} \|z - e_v\|_2, \quad (2.2)$$

where  $e_v$  are grid points learned rather than fixed so they concentrate where the data lives, and  $V$  is the size of the codebook. During training, the non-differentiable arg min is bypassed with the straight-through estimator plus a commitment term keeping encoder outputs close to their assigned codeword [33].

A single codebook trades off badly: halving the quantization error roughly squares the codebook size. *Residual vector quantization* (RVQ) [36] instead quantizes in several rounds, each codebook encoding what the previous ones missed. With  $r_t^{(1)} = z_t$  and a separate codebook  $\mathcal{C}_k$  per round  $k = 1, \dots, K$ , we end up with:

$$u_t^k = Q_{\mathcal{C}_k}(r_t^{(k)}), \quad r_t^{(k+1)} = r_t^{(k)} - e_{k, u_t^k}, \quad \hat{z}_t = \sum_{k=1}^K e_{k, u_t^k}. \quad (2.3)$$

The first codebook places a frame coarsely and each further one refines the remainder — coarse spectral content first, fine texture later, a hierarchy Section 2.2.3 exploits directly. In our case, the bitrate is  $R = f_r K \log_2 V = 50 \times 4 \times 11 = 2.2$  kbit/s for  $K = 4$ ,  $V = 2048$  at 50 Hz, against 512 kbit/s for the raw waveform.

EnCodec [10] instantiates this recipe with a convolutional encoder–decoder and an RVQ bottleneck, trained with a multi-scale mel-spectrogram loss, an adversarial term, and the VQ commitment terms above. From here, the codec is a fixed, pretrained

interface: *a piece of audio is a grid of tokens*

$$U = (u_t^k)_{t=1..T}^{k=1..K}, \quad u_t^k \in \{1, \dots, V\}, \quad (2.4)$$

with  $T = 50\tau$  frames of  $K = 4$  tokens each. Generating music means generating such a grid and decoding it; editing means regenerating part of one. Next section builds the language model that speaks it.

## 2.2 Autoregressive generation: MusicGen

### 2.2.1 Music as a distribution over sequences, learned by MLE

Generative modeling of music postulates that clips — here, token grids  $U$  — are samples from an unknown distribution  $p^*$ , and seeks a model  $p_\theta$  that approximates  $p^*$  closely enough that samples  $U \sim p_\theta$  are perceptually musical. The central obstacle is combinatorial scale. For a single token stream  $U = (u_1, \dots, u_T)$  with vocabulary size  $V = 2048$  and  $T = 1500$  (30 s of audio), the space of possible sequences has cardinality  $2048^{1500} \approx 10^{4966}$ . A distribution over such a space admits no explicit tabulation and must instead be represented implicitly, through a model that assigns probability on demand rather than by lookup.

The chain rule of probability provides the requisite structure. Repeated application of  $p(A | B) = p(A, B)/p(B)$  to a sequence yields the exact factorization

$$p(U) = p(u_1) p(u_2 | u_1) p(u_3 | u_1, u_2) \cdots = \prod_{t=1}^T p(u_t | u_{<t}), \quad (2.5)$$

where  $u_{<t} = (u_1, \dots, u_{t-1})$  denotes the prefix preceding position  $t$ . Equation (2.5) is an identity, not an approximation: it holds for any distribution over sequences and reduces the intractable problem of modeling  $p(U)$  directly to  $T$  tractable subproblems, each a distribution over a vocabulary of size  $V$  conditioned on the tokens generated so far. A model organized around this factorization can be termed as *autoregressive* (AR).

Rather than storing  $T$  separate conditionals, a single network — in the lineage of neural language models initiated by Bengio et al. [5] — represents all of them jointly: given the past  $u_{<t}$ , it outputs logits  $\ell_t \in \mathbb{R}^V$ , which are converted into a categorical distribution over the vocabulary via the softmax function,

$$p_\theta(u_t = v | u_{<t}) = \text{softmax}(\ell_t)_v = \frac{\exp(\ell_{t,v})}{\sum_{v'=1}^V \exp(\ell_{t,v'})}. \quad (2.6)$$

Parameters are chosen by *maximum likelihood*: minimizing the negative average log-

likelihood, i.e. the cross-entropy

$$\mathcal{L}(\theta) = -\mathbb{E}_{U \sim p^*} \left[ \sum_{t=1}^T \log p_\theta(u_t \mid u_{<t}) \right]. \quad (2.7)$$

Training evaluates every conditional in (2.7) on ground-truth pasts (teacher forcing), so all  $T$  predictions are computed in one forward pass, provided the architecture guarantees position  $t$  never sees positions  $\geq t$  — the job of the causal attention mask, the object Chapter 3 replaces.

### 2.2.2 Transformer decoder and causal mask

MusicGen [9] implements the next-token predictor with a Transformer decoder [34]. Each token is embedded to  $h_t \in \mathbb{R}^d$ , and a stack of layers transforms the sequence via *self-attention*: from each  $h_i$ , three projections give a query  $q_i$ , key  $k_i$ , value  $v_i$ , and position  $i$  gathers information through a weighted average comparing its query to every key,

$$\alpha_{ij} = \text{softmax}_j \left( \frac{q_i^\top k_j}{\sqrt{d}} + M_{ij} \right), \quad o_i = \sum_j \alpha_{ij} v_j. \quad (2.8)$$

Multiple heads run in parallel, and each layer adds a feed-forward network and residual connections.

The matrix  $M$  is the *attention mask*, entries in  $\{0, -\infty\}$ : adding  $-\infty$  before the softmax zeroes that weight. Autoregressive training uses the causal (lower-triangular) mask

$$M_{ij}^{\text{causal}} = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i, \end{cases} \quad (2.9)$$

so position  $t$ 's output legitimately parameterizes  $p_\theta(u_{t+1} \mid u_{\leq t})$ . Causality thus lives not in the (order-agnostic) layers but in one additive matrix injected into every attention operation — the seed of Chapter 3: editing is obtained by changing  $M$ , not the network.

### 2.2.3 One step, four codebooks: the delay pattern

The chain rule above assumed one token per step, but the codec emits  $K = 4$  per frame. *Flattening* into one stream of length  $KT$  preserves exact AR factorization but multiplies generation time by four; predicting all  $K$  in parallel keeps the sequence short but wrongly assumes them conditionally independent, when fine codebooks refine exactly what coarse ones chose (Section 2.1.1). MusicGen's *delay pattern* [9] solves this problem: codebook  $k$  is delayed by  $k - 1$  steps, e.g

$$\tilde{u}_s = (u_s^1, u_{s-1}^2, u_{s-2}^3, u_{s-3}^4), \quad (2.10)$$

padded where indices run out of range, unrolling over  $T + K - 1$  steps. When the model emits fine token  $u_t^k$ , the coarser tokens of the *same* frame are already in the visible past, so RVQ’s coarse-to-fine structure is respected at a cost of only  $K - 1$  extra steps, matching flattening in quality at a quarter of the compute [9]. An “edit window” in Chapter 3 is a window in this *delayed* coordinate system, and the mask must account for it.

### 2.2.4 Conditioning: text enters through cross-attention

Practical systems model  $p_\theta(U \mid c)$ ,  $c$  a free-form text description encoded by a frozen T5 [26] into vectors  $C = (c_1, \dots, c_L)$ . Every decoder layer gets a second attention block, *cross-attention*, identical to (2.8) except queries come from the decoder while keys/values come from  $C$ :

$$\alpha_{ij}^x = \text{softmax}_j \left( \frac{q_i^\top k(c_j)}{\sqrt{d}} \right), \quad o_i^x = \sum_{j=1}^L \alpha_{ij}^x v(c_j). \quad (2.11)$$

The same mechanism accommodates any condition shaped as a sequence of vectors — melodies and audio prompts can enter through this door too (Section 2.4).

### 2.2.5 Decoding: sampling, KV cache, and guidance

Generation unrolls (2.5) into a loop: predict, sample, append, repeat. Greedy decoding collapses onto bland, repetitive material [20], so a temperature  $\tau_s > 0$  reshapes the distribution instead,

$$p_v^{(\tau_s)} \propto p_v^{1/\tau_s}, \quad (2.12)$$

and top- $k$  truncates to the  $k$  most probable tokens before renormalizing; MusicGen uses  $\tau_s = 1$ ,  $k = 250$ , but parameters can be tuned differently at inference.

Naively, predicting token  $t$  costs quadratically in sequence length per generated token. The *KV cache* removes this: since the causal mask prevents position  $j$  from seeing anything after  $j$ , past keys/values never change as generation proceeds, so they are computed once and reused, an *exact* consequence of the mask’s structure. An editing method should preserve this property.

*Classifier-free guidance* (CFG) [19] counters conditional models following their prompt too loosely. The condition is randomly dropped during training (10% in MusicGen), so one network learns both  $p_\theta(\cdot \mid c)$  and  $p_\theta(\cdot \mid \emptyset)$ . At sampling time, both are evaluated and extrapolated with scale  $\gamma \geq 1$ ,

$$\ell_t^{\text{cfg}} = \ell_t(\emptyset) + \gamma (\ell_t(c) - \ell_t(\emptyset)). \quad (2.13)$$

$\gamma = 1$  recovers the conditional model; larger  $\gamma$  adheres more tightly at some diversity cost. MusicGen uses  $\gamma = 3$ . Acting purely on output logits, as this method relies only at inference, it composes freely with any change to the attention structure (Chapter 3).

## 2.3 Editing & infilling: state of the art

### 2.3.1 Infilling as conditional probability

Let partition the sequence  $U$  into three parts  $U = (P, E, S)$ . The task of editing samples a new window from

$$p(E \mid P, S) = \frac{p(P, E, S)}{p(P, S)}, \quad (2.14)$$

conditional probability’s ordinary definition: among complete sequences beginning with  $P$  and ending with  $S$ , renormalize the windows in between. A window drawn this way is coherent with both past and future by construction. Factoring the joint and cancelling  $p(P)$ ,

$$p(E \mid P, S) = \frac{p(E \mid P) p(S \mid P, E)}{\sum_{E'} p(E' \mid P) p(S \mid P, E')}. \quad (2.15)$$

An edit should be a plausible continuation of  $P$  and make  $S$  plausible in turn. A causal model evaluates both factors but cannot *sample* from their product: generating left-to-right, it draws from  $p(E \mid P)$  alone, blind to the landing term. Indeed, both factors in (2.15) are individually computable by teacher-forced scoring —  $p(E \mid P)$  by scoring  $E$  given  $P$ , and  $p(S \mid P, E)$  by scoring  $S$  given  $P$  and a candidate  $E$  — but left-to-right *sampling* has no such access: at the moment  $E$  is drawn,  $S$  is not yet part of the causal context, so the model cannot weight candidates by how well they set up the suffix. And the denominator sums over  $V^{K|E|}$  candidates. The field has answered in three ways: leave the AR paradigm for masked iterative decoding, leave it for diffusion, or keep it and rearrange the sequence.

### 2.3.2 Masked iterative decoders

MaskGIT [8] abandons left-to-right factorization for masked prediction: a random subset  $\mathcal{M}$  is replaced by [MASK], and a fully-visible Transformer (no causal mask) learns

$$\mathcal{L}(\theta) = -\mathbb{E}_{U, \mathcal{M}} \left[ \sum_{i \in \mathcal{M}} \log p_{\theta}(u_i \mid U_{\setminus \mathcal{M}}) \right]. \quad (2.16)$$

This is a native infiller, but generation iterates: predict all masked positions in parallel, keep the most confident, re-mask the rest, repeat — each round assuming conditional independence among masked tokens, false in general but mitigated by more rounds.

VampNet [15] brings this to music with flexible masking; MAGNeT [37] adds span masking and codebook-by-codebook decoding, reporting quality near MusicGen at a fraction of the inference steps. The price: the exact factorization (2.5), single-pass decoding and exact KV caching, with quality now depending on the refinement schedule.

### 2.3.3 Diffusion models and inpainting

Diffusion models instead model the codec’s continuous latents directly, before quantization. A forward process corrupts data with Gaussian noise,

$$q(z_s | z_0) = \mathcal{N}(\sqrt{\bar{\alpha}_s} z_0, (1 - \bar{\alpha}_s) I), \quad (2.17)$$

$\bar{\alpha}_s$  decreasing  $1 \rightarrow 0$ ; a network learns to invert it step by step, so sampling denoises from pure noise. Stable Audio [13] is the reference system for music. Inpainting is nearly free: at each step, kept-region coordinates are overwritten with a noised copy of ground truth from (2.17), pinning the trajectory to known context while the window is free — strong global coherence, every pass sees the whole clip, but a fixed budget of full-sequence passes regardless of edit size, with no counterpart to KV caching or per-token control.

### 2.3.4 Keeping the AR model: infilling by rearrangement

Text and code modeling keep the causal decoder and *rearrange the training data*. Fill-in-the-middle (FIM) [4] moves the window to the end,

$$([\text{PRE}], P, [\text{SUF}], S, [\text{MID}], E), \quad (2.18)$$

so predicting  $E$  left-to-right conditions, via ordinary causal attention, on both  $P$  and  $S$ , now preceding it: sampling from  $p(E | P, S)$  becomes plain next-token generation. Donahue et al. [12] and InCoder [14] realize the same idea; Bavarian et al. [4] show the capability comes essentially free for text. In symbolic music, the Piano Inpainting Application [18] applies it to note events.

Transposing this to codec grids is less innocent: audio tokens live on a rigid temporal lattice, staggered by the delay pattern (2.10), and moving the suffix before the window breaks the correspondence between position and musical time — positions no longer measure elapsed time, and delayed codebooks straddle the cut. This would be possible but nothing would precisely keep the time condition. In this work, we instead keep every token where time puts it and changes *visibility*, not *order*: since causality lives in the mask (2.9), the mask itself can be reshaped so the window sees both sides of its

| Family                              | Training objective     | Decoding    | Infilling | KV cache |
|-------------------------------------|------------------------|-------------|-----------|----------|
| Causal AR (MusicGen)                | next token             | 1 pass, L→R | no        | exact    |
| Masked iterative                    | masked prediction      | $R$ rounds  | native    | no       |
| Latent diffusion                    | denoising              | $S$ steps   | native    | no       |
| AR + FIM                            | next token (reordered) | 1 pass, L→R | native    | exact    |
| <b>AR + span mask (this report)</b> | next token (edit-mode) | 1 pass, L→R | native    | exact    |

Table 2.1: Families of approaches to music-audio editing. “L→R” denotes left-to-right generation;  $R$  and  $S$  are iteration budgets chosen at inference time.

context while the context stays causal (Chapter 3).

### 2.3.5 Positioning

Table 2.1 summarizes the landscape: each family gives something up. Masked iterative decoders give up exact factorization and single-pass decoding; diffusion gives up the token-level AR toolbox; FIM keeps the decoder but disturbs the temporal lattice. This report keeps the AR factorization, decoding loop, caching and guidance untouched, and pays at training time with a modified attention mask and an edit-aware recipe.

## 2.4 Controllability

### 2.4.1 Three doors into the network

Editing answers *where* the model may act; control answers *what* it should do there, by enriching  $p_\theta(U \mid c)$  with further signals. The useful taxonomy is architectural: *where* does the signal enter?

1. **Sequence prepending.** Tokens or embedding frames placed before the generated sequence, read via ordinary self-attention. Natural for signals sharing the sequence’s temporal structure.
2. **Cross-attention.** A separate sequence consulted at every layer via (2.11). Natural for arbitrary length/modality; how MusicGen reads text.
3. **Feature-wise modulation.** Compress the signal to one vector that rescales and shifts activations. Natural for a *global* attribute — a style, a class, a corpus.

Chapter 3 uses all three: chromagrams via door 1, CLAP via door 2, the dataset token via door 3.

### 2.4.2 Melody and CLAP: chromagram and joint embedding

“Follow this melody” needs a representation ignoring what should stay free — timbre, octave, accompaniment. The *chromagram* folds the short-time Fourier transform’s

frequency bins into the twelve pitch classes:

$$\text{pc}(f) = \left\lfloor 12 \log_2(f/f_{\text{ref}}) \right\rfloor \bmod 12, \quad C_{t,q} = \sum_{f: \text{pc}(f)=q} |X(t, f)|, \quad (2.19)$$

giving a 12-dimensional time series where a melody line is a moving ridge, identical regardless of timbre or octave. MusicGen’s melody variant [9] conditions on the dominant pitch class per frame (argmax of (2.19)), prepended alongside text. The same representation, restricted to the edit window, dictates melody in Section 3.2.2.

Contrastive language–audio pretraining (CLAP) [35] trains an audio and a text encoder jointly so matched pairs land close in a shared space and mismatched pairs far apart. With normalized embeddings  $a_i$  (audio),  $e_j$  (text) and temperature  $\tau_c$ , the symmetrized InfoNCE objective over a batch of  $B$  pairs is

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(a_i^\top e_i / \tau_c)}{\sum_{j=1}^B \exp(a_i^\top e_j / \tau_c)} + (\text{text} \rightarrow \text{audio}), \quad (2.20)$$

a batch-wise classification problem. A conditioner trained on CLAP embeddings accepts either modality at inference — text and audio are just two points in one space — so an existing recording can directly describe a desired sound world.

### 2.4.3 Global attributes: FiLM and adaptive layer normalization

The third door modulates activations. Layer normalization [3] standardizes each hidden vector and re-dresses it with learned scale/shift,

$$\text{LN}(h) = g \odot \frac{h - \mu(h)}{\sigma(h)} + b, \quad \mu(h) = \frac{1}{d} \sum_i h_i, \quad \sigma^2(h) = \frac{1}{d} \sum_i (h_i - \mu(h))^2. \quad (2.21)$$

FiLM [25] turns  $g, b$  into functions of the condition,  $\gamma(c) \odot h + \beta(c)$ ; *adaptive layer normalization* (AdaLN), popularized by diffusion Transformers [24], applies this at every normalization site:

$$\text{AdaLN}(h; c) = (1 + \gamma(c)) \odot \frac{h - \mu(h)}{\sigma(h)} + \beta(c). \quad (2.22)$$

Zero-initialized per-block gates make the conditioned network coincide with the unconditioned one at initialization. This door suits a *global, discrete* attribute: uniform across time steps, and parsimonious, one learned vector per class, negligible extra parameters. Section 3.2.3 uses it for a one-bit dataset token choosing pop vs. Schaeffer at edit time.

## 2.5 Multi-stem generation and MusicGen-Stem

Every model above treats a clip as one waveform, so none can redo *one instrument* inside a window while leaving the others alone (Section 1.3). Closing this gap needs a second axis of decomposition alongside the frame axis of Section 2.1, and MusicGen-Stem [29] supplies it: a source separator splitting a mix into stems, plus a tokenization scheme letting one language model read and write all of them.

### From one signal to several: source separation

Hybrid Transformer Demucs [27] (`htdemucs`), the separator used throughout, is a U-Net operating jointly in the waveform and spectrogram domains with a cross-domain Transformer at its core, outputting one waveform per source — *drums*, *bass*, *vocals*, *other* — summing back to the input mix. It is a fixed, pretrained tool exactly as EnCodec was: a mix is deterministically mapped to four stems, each then tokenized on its own.

### Tokenizing several stems

MusicGen-Stem tokenizes each stem separately, at its own residual depth, and concatenates the codebooks into one wider grid the same Transformer reads unmodified. A single-stem model has  $K$  streams per frame; a 4-stem model has  $\sum_i K_i$ , with an unequal split following how much each stem’s residual actually carries: one coarse codebook each for the sparse *drums/bass*, four each for the harmonically dense *vocals/other*. Streams are ordered coarse-first *across* stems, so the existing delay pattern (2.10) generalizes with a single sorted delay vector. Nothing about the Transformer, the delay machinery, or the causal mask (2.9) needs to know some streams belong to different instruments: more stems are simply more codebooks, the same observation Section 2.2.2 made about causality, a structural property living in one auxiliary object rather than the architecture, and what lets Chapter 3 extend the asymmetric mask to stems unchanged.

### MusicGen-Stem’s own editing task

Because a stem-tokenized clip lets a subset of *streams* be frozen as easily as a subset of frames, MusicGen-Stem’s own recipe already includes a form of editing with no mask change: with some probability, mask one or two stems’ tokens in a heavily downsampled coarse prefix, then train ordinary next-token prediction on the full-resolution block, the frozen streams simply copied from their coarse hint. The result is a causal model that regenerates a chosen subset of stems given the rest — Section 2.3.4’s *rearrange the input* strategy, applied to *which stem* rather than *which time step* — but only ever

over a whole clip: nothing lets the frozen stems extend past the context window, or a sub-window of one stem be touched. Chapter 3 instead reuses the one mask this report already has, composed with a stream selection, for the stem axis, the time axis, and both together.

The pieces are on the table: a codec, a causal Transformer whose causality lives entirely in one mask, the vacant niche in the editing landscape, three conditioning doors, and a stem axis the same Transformer reads as more codebooks. Chapter 3 builds the editor from these pieces, ending with the one composing all of them: a stem-by-time editor that regenerates a chosen instrument inside a chosen window.

# Chapter 3

## Method

Chapter 2 showed that every existing family of editing methods buys infilling at the cost of some other property (Table 2.1). This chapter describes a method that does not. Section 3.1 turns a causal MusicGen-style decoder into a sampler for  $p(E \mid P, S)$  by changing one component — the attention mask — together with the training procedure that goes with it; the architecture, the decoding loop and classifier-free guidance (CFG) are left unchanged. Section 3.2 adds the controls a composer needs on top of that mechanism: a scalar weighting the two sides of the context, a target-melody input, and a target-timbre input. Section 3.3 combines both with MusicGen-Stem, so that a single instrument can be edited inside a time window, and chains the result into pieces longer than one context window.

### 3.1 Editing as infilling

In this section, we describe the new attention mask used to make infilling possible in AR models while keeping the temporal structure and the KV cache.

#### 3.1.1 Problem formulation

Fix a clip of  $T$  audio frames and its token grid  $U = (u_t^k)$ ,  $t = 1..T$ ,  $k = 1..K$ . An edit request partitions frame indices into a prefix  $P = [1, a)$ , an edit window  $E = [a, b)$ , and a suffix  $S = [b, T]$ ; editing means sampling

$$p_\theta(E \mid P, S), \tag{3.1}$$

the conditional of Section 2.3.1: by (2.15) it rewards windows that continue  $P$  plausibly *and* make  $S$  plausible, but a causal model cannot sample from their product without seeing  $S$  during generation. The approach taken here pays a different price from the three families of Section 2.3: the causal decoder is kept, and only what the edit window

is *allowed to see* changes. Concretely, the system must satisfy four requirements:

- (R1) generation inside  $E$  remains strictly left-to-right, so sampling, truncation and CFG carry over unchanged;
- (R2) the hidden states — hence the KV cache — of  $P$  and  $S$  are independent of what is generated in  $E$ ;
- (R3) ordinary causal generation is recovered *exactly* as a special case; and
- (R4) the mechanism composes with further, independently switchable control signals.

All four follow from a single observation: causality in a Transformer is not a property of its (order-agnostic) layers but of the additive matrix  $M$  injected into every attention operation (2.8). Section 3.1.2 replaces  $M^{\text{causal}}$  (2.9) with an asymmetric mask  $M^*$  satisfying (R1)–(R2) by construction and containing  $M^{\text{causal}}$  as the degenerate case giving (R3); Section 3.1.3 is the training recipe; Sections 3.2.1–3.2.3 are the control layer of (R4).

### 3.1.2 Asymmetric span attention mask

#### Definition

The mask lives in *pattern space*: the delay pattern (2.10) interleaves tokens from up to  $K$  frames per step, so the frame-space window  $[a, b)$  must first be transported there. Define

$$\phi(s) = \begin{cases} t & \text{if step } s \text{ carries a codebook-1 token from frame } t, \\ -1 & \text{if step } s \text{ carries no valid token,} \end{cases} \quad (3.2)$$

and classify steps by frame,

$$P_S = \{s : \phi(s) < a\}, \quad E_S = \{s : a \leq \phi(s) < b\}, \quad S_S = \{s : \phi(s) \geq b\}, \quad (3.3)$$

which partitions all steps into exactly one of  $P_S$ ,  $E_S$ ,  $S_S$ ; since  $\phi(s) \geq -1$  always, invalid steps ( $\phi(s) = -1$ ) fall into  $P_S$  automatically, for any  $a \geq 0$ .  $M^*$  is defined by the neighbor relation

$$\mathcal{N}(i) = \begin{cases} P_S & i \in P_S, \\ P_S \cup S_S & i \in S_S, \\ P_S \cup S_S \cup \{j \in E_S : j \leq i\} & i \in E_S, \end{cases} \quad M_{ij}^* = \begin{cases} 0 & j \in \mathcal{N}(i), \\ -\infty & \text{otherwise,} \end{cases} \quad (3.4)$$

or, in block form (rows queries, columns keys;  $\mathbf{1}/\mathbf{0}$  allowed/blocked,  $\triangle$  causal):

$$M^\star \longleftrightarrow \begin{array}{c|ccc} & P & E & S \\ \hline P & \mathbf{1} & \mathbf{0} & \mathbf{0} \\ E & \mathbf{1} & \triangle & \mathbf{1} \\ S & \mathbf{1} & \mathbf{0} & \mathbf{1} \end{array} \quad (3.5)$$

Two entries define the construction: the  $E$  column is  $\mathbf{0}$  on the  $P$  and  $S$  rows, so no position outside  $E$  can read it, and the  $S$  block of the  $E$  row is  $\mathbf{1}$ , so the window sees the whole future. Two further entries deserve emphasis because they are *not* causal: the  $P/P$  and  $S/S$  blocks are full, not triangular. The frozen context is therefore encoded bidirectionally and only  $E$  is decoded left to right, which makes the model an encoder and a decoder sharing one stack rather than a causal decoder with a relaxed mask. This costs nothing, since no context position is ever a loss target (Section 3.1.3), and it strictly increases the information available to  $E$ ; but it does mean that  $M^\star$  is not obtained from  $M^{\text{causal}}$  by a single off-diagonal flip, and that the degenerate cases in which the two coincide are exactly those in which  $P_S$  and  $S_S$  are empty.

The classification (3.3) is exact only up to the skew introduced by the delay pattern: a step just past the  $E_S/S_S$  boundary can still carry, on a higher codebook, a token from a frame inside  $[a, b)$ , since a codebook at delay  $k - 1$  places frame  $t$ 's content at step  $t + k - 1$  rather than step  $t$ . The properties below are proved with respect to the step-space partition  $(P_S, E_S, S_S)$  and hold there exactly. Translated to frame space they hold only up to a skew of at most  $K - 1$  frames at each boundary: the  $K - 1$  steps immediately after  $b$  are classified  $S_S$  yet carry deep-codebook tokens of frames inside  $[a, b)$ . Padding  $b$  by  $K - 1$  would remove the discrepancy at the cost of freezing  $K - 1$  additional frames; the implementation does not pad, and Sections 3.1.3 and 3.1.3 state the consequences.

### Two properties, and why not the symmetric mask

Throughout, we assume a standard decoder-only transformer: the mask  $M^\star$  is applied identically at every layer, cross-position mixing occurs only through masked attention (normalization and feed-forward sublayers act position-wise), and each position's keys and values are a deterministic function of that position's own hidden state at the previous layer.

**Property 1** (Context independence). *For every layer  $\ell$  and  $i \in P_S \cup S_S$ ,  $x_i^{(\ell)}$  under  $M^\star$  depends only on input tokens in  $P_S \cup S_S$ .*

This is requirement (R2), made precise. *Proof.* By induction on  $\ell$ . At  $\ell = 0$ ,  $x_i^{(0)}$  is a function of input  $i$  alone. Assume the claim at layer  $\ell$ . For  $i \in P_S \cup S_S$ , (3.4) gives  $\mathcal{N}(i) \subseteq P_S \cup S_S$  (equal to  $P_S$  or  $P_S \cup S_S$  respectively), so  $x_i^{(\ell+1)}$  is computed from

$\{x_j^{(\ell)} : j \in \mathcal{N}(i)\} \subseteq \{x_j^{(\ell)} : j \in P_S \cup S_S\}$ , each of which depends only on context inputs by the inductive hypothesis.  $\square$

Consequently, the keys and values at  $P_S \cup S_S$  never change no matter what is sampled at  $E_S$ : they depend only on context inputs at every layer, and can be cached *exactly*, not approximately. The statement is a step-space one, and the frame-space skew noted above is the price: the  $K - 1$  steps after  $b$  belong to  $S_S$  but are fed deep-codebook tokens of edit frames, so their keys and values do change with the edit. Freezing the suffix cache in frame space therefore requires padding  $b$  by  $K - 1$ ; at  $K = 4$  and 50 Hz this is 60 ms.

**Property 2** (Left-to-right factorization inside  $E$ ). *For  $i \in E_S$ ,  $\mathcal{N}(i) \cap E_S = \{j \in E_S : j \leq i\}$ , so  $x_i^{(\ell)}$  depends on all of  $P, S$  together with the causal past of  $E$  alone, and*

$$p_\theta(E \mid P, S) = \prod_{i \in E_S} p_\theta(e_i \mid e_{<i}, P, S). \quad (3.6)$$

Two facts combine to give (3.6). The neighbor structure of  $E_S$  in (3.4) fixes the *order*: each  $i \in E_S$  conditions on exactly the earlier edit positions  $e_{<i}$ , matching the model’s autoregressive sampling order along the delay-pattern step sequence. Property 1 fixes the *well-foundedness*: it guarantees  $P$  and  $S$ ’s hidden states are exogenous to  $E$ , so conditioning on “ $S$ ” in (3.6) denotes a fixed quantity rather than one entangled with the variable being sampled. Without Property 1, the right-hand side would not be a well-defined chain-rule factorization. This is requirement (R1).

Both properties rest on the same asymmetry. Were  $S$  allowed to attend into  $E$ , Property 1 would fail for  $S_S$ , the induction would break, and the hidden states would become a coupled fixed point with no constant context to cache, resolvable only by iterative refinement (Section 2.3.2). Admitting the edge  $E \rightarrow S$  but not its reverse is thus what separates a model that remains autoregressive and cacheable from a non-autoregressive one.

## Implementation and empirical validation

Injecting  $M^*$  requires two changes to the reference implementation. First, the transformer must run non-causally, with  $M^*$  threaded through every attention call. Causality is a masking behavior rather than a learned quantity, so this changes no tensor shape and a pretrained checkpoint loads unchanged if desired (Section 3.1.3). Second, the mask must be honored on its numerical values rather than its presence. The memory-efficient attention path of the MusicGen reference implementation does not propagate an additive mask when it runs on the PyTorch scaled dot-product backend: it calls that kernel with the boolean flag `is_causal` set according to whether a mask was supplied, and discards the mask tensor itself. A model trained through that path

| Mode                | Window                                                | Resulting form of $M^*$                                 |
|---------------------|-------------------------------------------------------|---------------------------------------------------------|
| <b>interior</b>     | $a \sim \mathcal{U}\{\gamma, T-L-\gamma\}, b = a + L$ | asymmetric: $P, S \neq \emptyset$                       |
| <b>continuation</b> | $a = T - L, b = T$                                    | $S = \emptyset$ ; $E$ causal over a bidirectional $P$   |
| <b>prefix_less</b>  | $a = 0, b = L$                                        | $P = \emptyset$ ; $E$ still attends to all of $S$       |
| <b>full_causal</b>  | $a = 0, b = T$                                        | $P = S = \emptyset \Rightarrow M^* = M^{\text{causal}}$ |

Table 3.1: The four edit modes sampled during training, with window length  $L$  and minimum per-side context  $\gamma$ ;  $a, b, L$  and  $\gamma$  are audio-frame counts obtained from the configured durations at the frame rate  $f_r$ , and  $\mathcal{U}\{\cdot, \cdot\}$  denotes the discrete uniform distribution. Windows that cannot be placed within the segment fall back to **full\_causal**. Configuration used throughout:  $\pi = (0.60, 0.15, 0.05, 0.20)$ ,  $l_{\min} = 3\text{ s}$ ,  $l_{\max} = 10\text{ s}$ ,  $\gamma = 1\text{ s}$ , 10-second segments. Because **interior** additionally requires  $\gamma$  of context on each side, its window length is clipped above at  $T - 2\gamma = 8\text{ s}$ , so roughly 29% of interior windows sit at that ceiling and cover 80% of the clip.

therefore receives an ordinary causal mask regardless of  $M^*$ , without raising an error and without any signature in the loss curve. All experiments in this report consequently use the manual attention path, which computes  $QK^\top/\sqrt{d} + M^*$  before the softmax as in (2.8).

The construction is also checked empirically, by unit tests asserting that  $M^*$  coincides with  $M^{\text{causal}}$  in the full-window case; that no query step is left without at least one allowed key; that every step assigned to  $E_S$  maps under  $\phi$  to a frame inside  $[a, b]$ ; that no  $P$  or  $S$  query attends to an  $E_S$  key, which is the structural form of Property 1; and that the decoded window is invariant, bit for bit, to whatever tokens originally occupied  $[a, b]$ , which is its end-to-end consequence.

### 3.1.3 Training: edit-mode recipe on small compute

#### The edit-mode mixture

The mask  $M^*$  makes infilling representable, but the pretrained checkpoint has only ever been exposed to  $M^{\text{causal}}$ ; converting it into an editor is therefore a question of training recipe. At every step a window is drawn hierarchically: first a mode  $m \sim \text{Categorical}(\pi)$  over four possibilities, then a window consistent with that mode.

The modes **continuation** and **full\_causal** are not auxiliary tasks introduced alongside the objective; they are  $M^*$  evaluated at  $S = \emptyset$  and at  $P = S = \emptyset$  respectively. Only the second collapses to  $M^{\text{causal}}$ , and it is that case which instantiates requirement (R3): with the whole clip inside  $E$  there is no context block left to encode bidirectionally, and  $M^*$  is lower triangular by construction. **continuation** is the ordinary autoregressive task with one difference, namely that its prefix is read bidirectionally; keeping it in the mixture is what lets the same checkpoint extend a clip as well as repair one. Because **full\_causal** reproduces a plain causal forward pass exactly, allocating a fraction  $\pi_{\text{full_causal}}$  of every step to it anchors the model against

drift away from unconditional left-to-right generation — and, when the recipe is used to fine-tune rather than to train from scratch, against catastrophic forgetting of the pretrained behaviour. The remaining mode, `prefix_less`, has no analogue in ordinary generation, since  $P = \emptyset$  while  $E$  still attends forward into  $S$ . It is weighted lowest and is included to keep the model from assuming that left context is always available.

### Loss restricted to the edit window’s causal zone

The cross-entropy objective (2.7) is a valid next-token loss only at positions where the model predicts from its own causal past. Under  $M^\star$  this holds inside  $E$  alone: by (3.5), positions in  $P_S$  and  $S_S$  attend bidirectionally within the context, so their outputs are not next-token predictions and must not be graded.

The restriction must be applied by pattern step rather than by frame. Codebook  $k$ ’s token for frame  $t$  occupies step  $s = t + (k - 1)$ , so that token lies in the causal zone of  $E$  if and only if

$$\mathcal{M}(a, b) = \{(k, t) : a \leq t + (k - 1) < b\}, \quad (3.7)$$

where  $a$  and  $b$  are the frame-space window boundaries of Section 3.1.1 and  $t + (k - 1)$  is read as the codebook-1 frame of the step that token  $(k, t)$  occupies. Selecting by frame instead would mis-grade the deep-codebook tokens that the delay displaces across each seam.

Two consequences are worth making explicit, because they are the practical residue of the frame/step skew. First, the set of frames with at least one graded token is  $[a - K + 1, b)$ , not  $[a, b)$ : at the leading boundary the delay pulls the *fine* codebooks of the  $K - 1$  frames before  $a$  into  $E_S$ , and at the trailing boundary it pushes the fine codebooks of the  $K - 1$  frames before  $b$  out of it. The graded set is thus a parallelogram, coarse-shifted-late at one edge and fine-shifted-early at the other, rather than a rectangle. Second, and less benignly, the graded set and the *generated* set do not coincide: at inference (Section 3.1.3) it is exactly frames  $[a, b)$  that are resampled, so the fine codebooks of the last  $K - 1$  edit frames are decoded from query steps that training never graded, and that  $M^\star$  classifies as  $S_S$  — steps which, by (3.4), cannot attend to any position in  $E$ . At  $K = 4$  this affects the deepest residual levels of the final 3 frames of every edit, 60 ms at 50 Hz. Padding the window by  $K - 1$  frames on the trailing side, or grading the loss on the union of the two sets, would align them; neither is done in the runs reported here, and Section 5.2.2 returns to it.

The training loss is (2.7) restricted to  $\mathcal{M}(a, b)$ :

$$\mathcal{L}_{\text{edit}}(\theta) = -\mathbb{E} \left[ \frac{1}{|\mathcal{M}(a, b)|} \sum_{(k, t) \in \mathcal{M}(a, b)} \log p_\theta(u_t^k \mid u_{<s}, S) \right], \quad s = t + (k - 1), \quad (3.8)$$

where  $u_{<s}$  denotes all tokens occupying pattern steps strictly earlier than  $s$  — which already includes the whole of  $P$ , so only  $S$  need be named separately. The  $K$  tokens sharing a step are predicted in parallel and are conditionally independent given  $u_{<s}$ , so no ordering among codebooks within a step is implied. For the `full_causal` mode,  $\mathcal{M}(a, b)$  contains every valid token and (3.8) reduces to ordinary whole-sequence cross-entropy. Because the per-example normalization averages over graded positions only, windows of different lengths contribute comparably and no additional per-window reweighting is required; an optional per-mode weight  $\beta_m$ , left at 1 throughout, is available should the modes need to be decoupled from  $\pi$  in their gradient contribution.

### Training regime

Because setting `causal` to `false` changes no tensor shape, the implementation supports both fine-tuning from the public MusicGen-small checkpoint (300M parameters) and training from random initialization. The checkpoints evaluated on `shs` (Chapter 4) use the latter throughout. This is a deliberate design choice: the unconditional editing model, its melody-conditioned variant (Section 3.2.2), and a plain causal reference model are all trained from scratch on `shs`, so that differences between them are attributable to the training recipe rather than to differently pretrained starting points, up to run-to-run variance. Conditioning is left at  $\emptyset$  for the base editing model; the melody variant additionally removes the text cross-attention stack. Training runs on  $2 \times 48$  GB GPUs with float16 mixed precision, using a learning-rate-free Adam variant (D-Adaptation) under a cosine learning-rate schedule with 2000 warm-up steps, on 10-second segments; the batch size is 64 for the base editing model and 48 for the melody variant, whose prepended chroma block lengthens the sequence. The dataset-token model of Section 3.2.3 is likewise trained from scratch, on a larger and differently conditioned corpus. Training from scratch also avoids a mismatch that fine-tuning would introduce: because  $M^*$  encodes  $P$  and  $S$  bidirectionally (Section 3.1.2), a checkpoint pretrained under  $M^{\text{causal}}$  would see out-of-distribution context representations from the first step in every mode except `full_causal`. Fine-tuning from the public checkpoint remains available through the same flag and is a natural next step (Section 5.3), but is not the regime behind any result reported here.

### Inference: decoding an edit

At generation time the frozen context is encoded once and the edit window is filled left to right against it, following (3.6) and optionally under classifier-free guidance (2.13). One practical caveat follows from, rather than contradicts, Property 1: the suffix occupies positions in the future of the pattern sequence, so the streaming key-value cache used to accelerate ordinary causal decoding cannot be reused without

modification.

The present implementation instead recomputes a full forward pass over the entire sequence at each generated step. This is exact rather than approximate, and the two properties are what make it so: positions in  $E$  that have not yet been generated hold placeholder tokens, which are harmless because Property 2 prevents any  $E$  position from attending to a later  $E$  position, and Property 1 prevents  $S$  from attending into  $E$  at all. The cost is  $|E| + K - 1$  forward passes of  $O(T^2)$  attention each, against  $O(T^2)$  in total for cached causal decoding; the decode is therefore linear in the length of the edit rather than of the clip, which keeps short edits cheap but still leaves a factor  $|E|$  against a cached decoder. A cache-accelerated decoder that materializes  $K_P, V_P, K_S, V_S$  once and reuses them across steps is available in principle from Property 1 and is left as an engineering optimization (Section 5.2.2).

## 3.2 Controlling generation

The mask of Section 3.1.2 determines what the edit window may attend to, but not how a composer specifies what it should produce. This section adds three controls on top of it: a weighting of context by side (Section 3.2.1), a melody conditioning path (Section 3.2.2), and a selection of timbre through dataset conditioning (Section 3.2.3). Each is inactive at its default setting, so the model of Section 3.1.3 is recovered by construction when no control is exercised.

### 3.2.1 Steering the edit: a cosine context schedule

Under  $M^*$ , every position in  $E$  has equal access to  $P$  and  $S$ . This is structurally correct but uniform in time, whereas the relative importance of the two sides is not: a token generated near the start of the window is primarily a continuation of  $P$ , while one generated near the end must resolve onto  $S$ . The cosine context schedule reweights the two sides as decoding sweeps through the window (Figure 3.1). It is applied at inference time only, requires no retraining, and modifies no structural entry of  $M^*$ .

Let  $a_S = \min E_S$  and  $b_S = \max E_S + 1$  denote the step-space boundaries of the edit window induced by the frame window  $[a, b]$  through (3.3). Let  $t_i = (i - a_S)/(b_S - a_S) \in [0, 1]$  denote the generation progress at query step  $i \in E_S$ , and let  $\sigma_j \in \{0, 1\}$  indicate whether context key  $j$  lies in  $P_S$  or in  $S_S$ . Define the side weight

$$w(\sigma, t) = \begin{cases} \frac{1}{2}(1 + \cos \pi t), & \sigma = 0 \quad (P), \\ \frac{1}{2}(1 - \cos \pi t), & \sigma = 1 \quad (S), \end{cases} \quad (3.9)$$

so that  $w(0, \cdot)$  decreases from 1 to 0 and  $w(1, \cdot)$  increases from 0 to 1 as  $t$  runs from 0

to 1, with  $w(0, \frac{1}{2}) = w(1, \frac{1}{2}) = \frac{1}{2}$ . The two weights sum to 1 at every  $t$ , though this is a convenience of the parameterization rather than a constraint the mechanism needs: the softmax renormalizes, so only the ratio  $w(0, t)/w(1, t)$  affects the result. The weight is injected as an additive log-bias on edges that  $M^*$  already permits, with strength  $\alpha \geq 0$ :

$$\widetilde{M}_{ij}^* = M_{ij}^* + \alpha \log \max(w(\sigma_j, t_i), \varepsilon), \quad i \in E_S, j \in P_S \cup S_S. \quad (3.10)$$

Because  $\text{softmax}(s + c) \propto \exp(s) \exp(c)$ , the bias multiplies each already-allowed attention weight by  $w^\alpha$  before normalization. No entry that  $M^*$  sets to  $-\infty$  is altered, so the neighbor relation (3.4) is unchanged and Property 1 continues to hold: context positions are unaffected, since (3.10) applies only to queries in  $E_S$ . The floor  $\varepsilon > 0$  ensures the bias is a finite suppression rather than a hard block, so a key that the schedule down-weights to  $w = 0$  remains reachable; it also keeps (3.10) well defined at the endpoints, where  $w$  vanishes. At  $\alpha = 0$  the bias term is identically zero and (3.10) reduces exactly to (3.4), so the schedule is opt-in. Larger  $\alpha$  sharpens the crossover. Two caveats should be stated plainly. First, the schedule is off-distribution: training used hard  $\{0, -\infty\}$  masks and never a soft reweighting, so  $\alpha$  is the strength of an inference-time nudge rather than of a learned control, and nothing guarantees that quality varies monotonically with it. Second, near the endpoints the suppression is severe rather than gentle: at  $t \rightarrow 0$  every suffix key is floored at  $\varepsilon$  and biased by  $\alpha \log \varepsilon$ , which at  $\varepsilon = 10^{-3}$  and  $\alpha = 1$  is  $-6.9$  nats, closer to a block than to a nudge. Both are reasons the schedule is reported as future work rather than used in any result of Chapter 4. Under classifier-free guidance (2.13), the bias is applied identically in the conditional and unconditional passes, so that the guidance direction reflects the conditioning signal rather than a difference in context weighting. Like the mask itself, the schedule requires the value-honoring attention path of Section 3.1.2.

### 3.2.2 Musical control: melody conditioning

#### Prepending the target melody

Specifying a target melody requires a representation that fixes pitch content while leaving timbre, octave and accompaniment free. The argmax chromagram (2.19) provides one. Of the three conditioning mechanisms surveyed in Section 2.4.1, feature-wise modulation is ill-suited to a per-frame signal, being designed to carry a single global vector; Section 3.2.3 uses it for such a signal instead. Between the remaining two, this report follows MusicGen-melody [9] and prepends the chroma as a block of conditioning positions rather than summing it into the input embeddings frame by frame.

Three considerations motivate this choice. First, a summed signal supplies each position with the chroma of the frame it is predicting and nothing beyond it, whereas

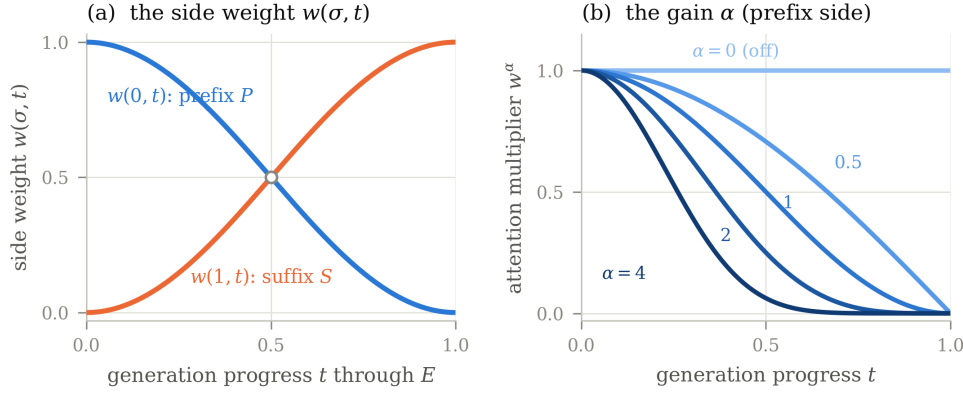


Figure 3.1: The cosine context schedule. **(a)** The side weight  $w(\sigma, t)$  of (3.9): the prefix weight falls from 1 to 0 as decoding sweeps through  $E$ , the suffix weight rises, and the two cross at  $\frac{1}{2}$ . The weight depends on which *side* a key lies on and on the progress  $t$ , never on the key’s absolute position in the clip. **(b)** The gain  $\alpha$  of (3.10) acts as an exponent on that weight, since an additive log-bias becomes a multiplicative factor  $w^\alpha$  after the softmax;  $\alpha = 0$  is the flat line, recovering the unmodified mask exactly. The bias is written only to the  $E_S$  rows over the  $P_S$  and  $S_S$  columns, so every structural  $-\infty$  of (3.4) is left untouched and Property 1 continues to hold.

a prepended block is visible in full from the first decoding step, so that the model can condition on the melodic contour ahead rather than on its current frame alone. Second, a prepended block need not be frame-aligned with the edit window, which allows a target melody of arbitrary length to be supplied at inference. Third, and most relevant to the present method, a prepended block can be made visible to a chosen subset of positions, whereas a summed signal enters the residual stream at every position it is added to and cannot be selectively withheld; Section 3.2.2 exploits this to keep the context representations independent of the melody.

The construction is self-supervised. At training time the chroma is computed from the ground-truth  $E$  segment, so the model is trained to regenerate a segment given  $P$ ,  $S$ , and a chromagram of the segment itself. MusicGen-melody first separates the clip with Hybrid Transformer Demucs [27] and takes the chroma of the summed **vocals** + **other** stems, so that drums and bass do not dominate the pitch estimate; that step is skipped here and the chroma is extracted from the mix directly. The trade is deliberate: separating every training segment with Demucs dominates the data-loading cost at this scale, and the resulting chromagram is noisier but identically distributed at training and at inference, since an arbitrary reference melody supplied at inference is likewise not separated. Section 3.3.1 recovers the clean version of the same signal for free, the stem model already holding the separated **other** stem in memory. At inference the chromagram of any target melody is substituted in its place. Reducing the chromagram to its argmax narrows the conditioning bottleneck, which limits how much of the reference beyond its pitch sequence can be recovered, and ensures that chromagrams computed from ground-truth audio at training time and from arbitrary references at

inference are of the same form. Following the classifier-free guidance formulation of Section 2.2.5, the chroma is dropped with probability  $p_{\text{chroma}}$  during training, leaving a null-chroma mode in which the model performs melody-free editing.

### Mask offset and conditioning visibility

Prepending  $L_c$  conditioning steps shifts every audio position by  $L_c$ , so that the classification (3.3) applies to the window  $[L_c + a, L_c + b)$  and  $M^*$  acquires a fourth block  $C = [0, L_c)$ .

The target melody is a known input, fixed in the same sense as  $P$  and  $S$ , so its presence does not by itself threaten Property 1. What the property does constrain is which positions may read  $C$ . Allowing every query to attend to  $C$ , as MusicGen-melody does, makes the hidden states at  $P$  and  $S$  functions of the target melody, so that their cached keys and values are invalidated whenever the melody changes. Restricting the readers of  $C$  to  $E_S$  leaves the context representations melody-independent: the induction of Property 1 goes through unchanged, since  $\mathcal{N}(i) \cap C = \emptyset$  for every  $i \in P_S \cup S_S$ . A single encoding of the context could then be reused across any number of candidate melodies for the same edit, which is the setting described in Section 1.1.

Both forms are implemented and selectable by a flag. The runs reported in Chapter 4 use the unrestricted form, in which  $P$  and  $S$  also read  $C$ : it is the MusicGen-melody behaviour, and the benefit of the restricted form is a cache reuse that the present decoder does not yet exploit (Section 3.1.3), so the two were not expected to differ at this stage. The restricted form is nonetheless the one to adopt once the cache-accelerated decoder exists, since that is the form the argument above justifies; measuring whether it costs anything in edit quality is left to future work.

Since  $L_c$  must be constant within a batch, the melody-conditioned model is trained with a fixed edit duration of 4s in place of the variable length of Table 3.1, and with a fixed 10% of steps drawn as the plain-causal anchor instead of the mixture  $\pi$ . This differs from the recipe of Section 3.1.3 in more than the melody conditioning alone — fixed rather than variable window length, one edit mode rather than four, and a different anchor rate — so the comparison in Chapter 4 is qualified accordingly.

### Timbre conditioning through the joint embedding

A chromagram specifies pitch content but not instrumentation or production. That is supplied separately, through cross-attention on a CLAP joint embedding (Section 2.4.2). The melody-conditioned editing model of Sections 3.2.2–3.2.2 is deliberately text-free and carries no cross-attention stack at all; the mechanism described here is the one used by the stem model of Section 3.3.1 and by the dataset-token model of Section 3.2.3, and is described at this point because it is the second half of the melody-plus-timbre

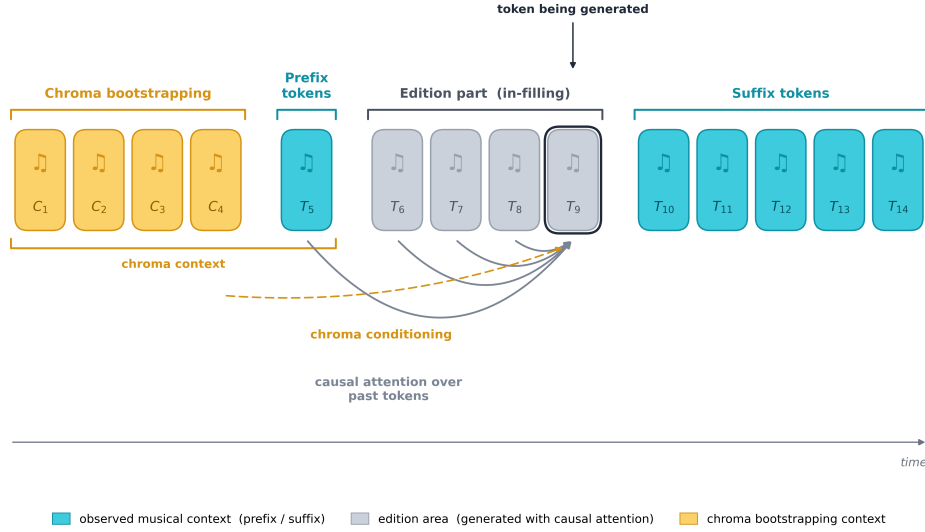


Figure 3.2: Span infilling in pattern space for a melody-conditioned edit. The chroma block  $C$  carrying the target melody is prepended; the audio positions then divide into the frozen prefix  $P$ , the regenerated window  $E$ , and the frozen suffix  $S$ . The window is filled one token at a time by causal attention over the already-generated positions, with every step additionally attending to the whole of  $C$ . Setting  $C = \emptyset$  and  $S = \emptyset$  recovers ordinary conditional generation; setting  $P = S = \emptyset$  recovers a prompt-prefixed decode.

pair that a composer specifies together.

Because text and audio describing the same sound are mapped to nearby points by the objective (2.20), a conditioner trained on one modality accepts the other at inference. At training time the condition is the CLAP audio embedding of the clip being reconstructed, so the setup is again self-supervised; at inference a text prompt is substituted. Since `shs` is essentially uncaptioned, the audio-embedding branch is what training actually sees on the stem model, and the text branch is exercised only at validation and generation time, where captions are used where they exist and the audio embedding is the fallback where they do not. The dataset-token model, whose corpus contains a captioned slice, instead rolls the text/audio choice per sample with probability  $\frac{1}{2}$ , so that captioned and uncaptioned material can be trained on together. The joint embedding is dropped with probability  $p_{\text{clap}}$ , independently of the chroma, so that all four combinations of present and absent conditioning are represented during training and the two controls can be applied independently at inference.

Routing the text condition through cross-attention follows MusicGen. The alternative, supplying it as prefix tokens within the same prepended block  $C$ , is equally well attested: it is used for music by MusicLM [2] and MusicGen-Style [28]. The present choice follows the base architecture rather than reflecting a measured difference between the two.

### 3.2.3 Style control: AdaLN dataset-token conditioning

#### A binary, global attribute

Which corpus a piece belongs to is, unlike a melody, a property that holds uniformly over a whole edit, which is the case feature-wise modulation is designed for (Section 2.4.3). It is carried here by a single label `dataset_source`  $\in \{\text{shs}, \text{schaeffer}\}$ . Because the vocabulary is closed, the label is embedded through an explicit lookup table rather than a hashed text embedding, with a dedicated learned *null* slot for a dropped or missing label so that CFG remains available on the flag itself.

Unlike melody and CLAP, the flag is not fused into the sequence or into cross-attention at all: it is routed directly to every layer’s AdaLN modulation, reusing (2.22) with  $c$  the label embedding. One projection per layer produces six vectors, giving the self-attention and feed-forward sub-blocks independent scale, shift and gate triples  $(\gamma_\ell, \beta_\ell, g_\ell)$ ,

$$h = \text{LN}(x) (1 + \gamma_\ell) + \beta_\ell, \quad x \leftarrow x + (1 + g_\ell) \cdot \text{sublayer}(h), \quad (3.11)$$

while the *cross*-attention sub-block, which carries the CLAP condition (Section 3.2.2), is left unmodulated, so that the corpus flag and CLAP remain separable control axes. The projection is zero-initialized, so at the start of training every AdaLN layer is exactly its unmodulated counterpart — the same fallback-to-identity property as the `full_causal` anchor and the  $\alpha = 0$  schedule. Note that  $\gamma_\ell$  and  $\beta_\ell$  here are the AdaLN scale and shift of (2.22), unrelated to the minimum context  $\gamma$  of Table 3.1 and the per-mode weight  $\beta_m$  of (3.8).

#### Why this mechanism suits this regime

The two classes are far from balanced: `shs` supplies thousands of hours against roughly thirty minutes of `schaeffer`, several orders of magnitude apart. Per-item manifest weights oversample the minority class so that both values are seen often enough to be learned. Feature-wise modulation is well matched to that regime: one embedding per class, refined through a single zero-initialized projection, needs far fewer gradient observations than a stylistic vocabulary threaded through cross-attention would. The flag and the CLAP embedding are dropped independently during training, with probabilities 0.1 and 0.3, so every combination is observed and both can be driven independently at inference.

The dataset-token mechanism was trained and evaluated on a text- and audio-conditioned generation model, without the asymmetric span mask active, since training a third model jointly exercising editing, melody, CLAP and the dataset flag was outside this internship’s compute budget. The two are nonetheless compatible by construction:

the forward pass reads the AdaLN condition from the same per-sample tensors whether or not an edit mask is supplied at that step, and the modulation acts position-wise, so it commutes with any choice of attention mask and nothing in Sections 3.1.2–3.1.3 would need to change for an edited window to carry a corpus flag. That the two compose *mechanically* is of course weaker than evidence that they compose *well*; Chapter 5 returns to what training all mechanisms on one model would take.

### 3.2.4 Adapting the joint embedding to a specialist vocabulary

The CLAP conditioner of Section 3.2.2 is used off the shelf. An earlier phase of this internship asked whether the joint embedding should instead be retrained when the conditioning vocabulary is a small, closed, expert one — the case `schaeffer` presents. The failure mode is the negative set: InfoNCE (2.20) treats every off-diagonal pair in a batch as a negative, which is sound for free-text captions and wrong for a closed vocabulary, where descriptors are polyhierarchical and two clips routinely share most of their label set, so much of the gradient separates pairs the annotation itself calls near-synonymous. Writing  $\mathcal{V}$  for the vocabulary and  $L_i \subseteq \mathcal{V}$  for the labels of item  $i$ , triplets are built per descriptor: for  $\ell \in L_i$  the anchor is the audio embedding  $a_i$ , the positive  $e_{i,\ell}^+$  the text embedding of an item whose labels contain  $\ell$ , the negative  $e_{i,\ell}^-$  one whose labels do not. With  $d(\cdot, \cdot)$  Euclidean, margin  $m$ , and  $\mathcal{L}_\Delta(a, e^+, e^-) = \max(0, d(a, e^+) - d(a, e^-) + m)$  [30], averaging within a label set before averaging over the corpus keeps heavily annotated items from dominating:

$$\mathcal{L}_{\text{MLTL}} = \frac{1}{N} \sum_{i=1}^N \frac{1}{|L_i|} \sum_{\ell \in L_i} \mathcal{L}_\Delta(a_i, e_{i,\ell}^+, e_{i,\ell}^{-,*}), \quad \mathcal{L} = \lambda_c \mathcal{L}_{\text{InfoNCE}} + \lambda_t \mathcal{L}_{\text{MLTL}}. \quad (3.12)$$

Negatives are drawn by semi-hard mining [30]. The triplet term is never used alone: it constrains relative distances only, so collapsing every item to one point satisfies it exactly, a degenerate optimum the training run duly found (Section 4.3.3); InfoNCE supplies the repulsion that keeps the space spread. The encoder is trained from scratch rather than fine-tuned from LAION-CLAP [35], its RoBERTa [23] body trained rather than frozen. The branch stopped there: the hybrid’s advantage over plain contrastive training was within what a ninety-eight-clip validation set can resolve, and the objective moved to the editing mechanism of Section 3.1, whose corpus has no label vocabulary.

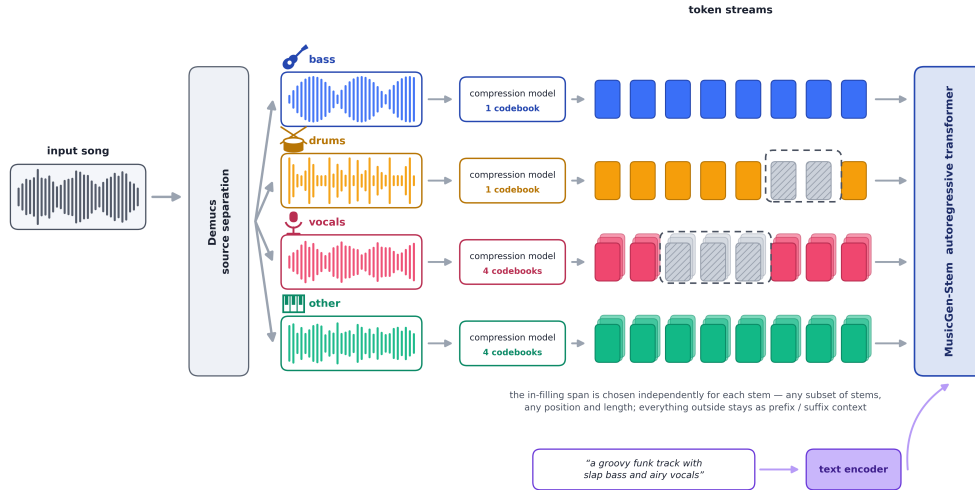


Figure 3.3: The stem model end to end. An input song is split by Hybrid Transformer Demucs into four stems (drums, bass, vocals, other), each tokenized by its own compression model — one codebook for bass and drums, four for vocals and other — into ten parallel token streams that a single autoregressive transformer models jointly, optionally conditioned on text. The in-filling span is chosen independently per stem — any subset of stems, any position and length — with everything outside it kept as frozen prefix/suffix context.

### 3.3 Stem-by-time editing

#### 3.3.1 Stem-by-time editing with MusicGen-Stem

Sections 3.1.2–3.2.3 edit a mix: the window  $E$  covers every stream. Section 2.5 supplied the missing axis — in a stem-tokenized clip, several instruments’ codebooks are concatenated into one set of streams — and the same mask  $M^*$ , composed with a selection of streams, edits a chosen instrument inside a chosen window without any change to the mask itself. This is the last of the models built during the internship, and the one combining the largest number of the preceding mechanisms (Figure 3.3; Sections 1.3, 1.4).

#### The base multi-stem generator

The starting point is MusicGen-Stem itself [29]: a causal decoder reading four stems — drums, bass, vocals, other — separated with Hybrid Transformer Demucs [27], each tokenized at its own residual depth ( $K_{\text{drums}} = K_{\text{bass}} = 1$ ,  $K_{\text{vocals}} = K_{\text{other}} = 4$ ,  $K = 10$  overall), arranged into a delayed token sequence with codebook  $k$  staggered by  $k - 1$  steps within its stem (Section 2.5; Figure 3.4). Three instantiation choices matter for Chapter 4.

**Per-stem codecs.** Drums, bass and other are each tokenized by a codec trained for that stem rather than by one shared EnCodec truncated to a single level; vocals

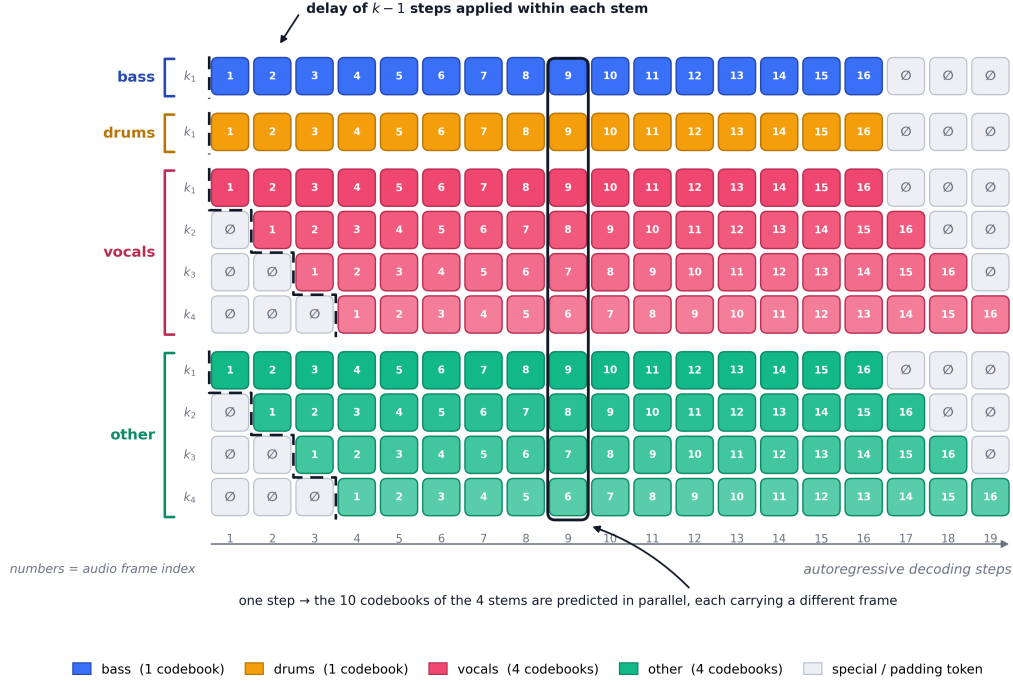


Figure 3.4: Delay pattern for the ten-stream MusicGen-Stem sequence, in the spirit of MusicGen’s own codebook-delay figure. Cell numbers are the audio *frame* a token encodes; the model predicts the sequence one *step* (column) at a time, emitting all ten codebooks of the four stems in parallel, each carrying a different frame. Within a stem, codebook  $k$  is delayed by  $k - 1$  steps so that finer levels are always predicted after coarser ones;  $\emptyset$  marks the special padding token filling the initial staggered slots. Bass and drums use a single codebook, vocals and other four each. The figure groups codebooks by stem for readability; internally the ten streams are reordered coarse-first (all level-0 codebooks, then all level-1, and so on) so the delay vector is sorted and the stock delayed-pattern provider applies unchanged.

reuse the shared full-music EnCodec, already in-distribution for singing. All codecs share cardinality, frame rate and sample rate, so their outputs concatenate regardless of origin.

**Chroma restricted to pitched stems.** The chroma conditioner reads the sum of *chosen* stems’ waveforms rather than the whole mix, so melody control targets the instrument it is meaningful for and unpitched drums is excluded by construction. *Other* is the target stem throughout Chapter 4.

**A loss weight for the bass stem.** A stream that is frequently silent, bass, in much of this corpus, contributes little loss whether or not it is predicted well, diluting its gradient. Rather than give bass a second codebook, this report re-weights its cross-entropy term:

$$\mathcal{L}(\theta) = -\mathbb{E} \left[ \frac{1}{\sum_k w_k} \sum_{k=1}^K w_k \log p_{\theta}(u_t^k \mid u_{<(k,t)}) \right], \quad w_k = \begin{cases} 2 & k \in \text{bass's stream(s)}, \\ 1 & \text{otherwise,} \end{cases} \quad (3.13)$$

amplifying bass’s gradient on clips that contain it, with no new stream or tokenization change. A stream with no graded positions is skipped rather than contributing a NaN.

**Training the base generator.** MusicGen-Stem’s own recipe (Section 2.5) is followed: with probability  $p_{\text{regen}} = 0.5$ , one or two stems are masked in a factor-5 downsampled (10 Hz) coarse prefix and the model reconstructs the full-resolution clip on every stream; otherwise plain text-and-chroma generation trains with no prefix. Two details depart from the published description. A dedicated transition token marks the boundary between the coarse prefix and the full-resolution block, so the model need not infer the change of frame rate from content alone. And when a masked stem has several codebooks, a random suffix of its residual levels is masked ( $\{4\}$ ,  $\{3, 4\}$ , ...) rather than all of them, which turns a fraction of the regeneration steps into a coarse-to-fine refinement task. This stage is purely causal, from scratch, on 10-second segments.

### The composed mask and loss

Editing one stem starts from the fine-tuned base generator and reuses Sections 3.1.2–3.1.3 unchanged, plus one addition. A step’s hidden state sums that step’s  $K$  token embeddings, and  $M^*$  classifies steps by frame alone — it has no notion of *which stream* a token belongs to. Editing a stem therefore needs no new mask, only a restriction on which streams count as “inside  $E$ .” Let  $R \subseteq \{1, \dots, K\}$  be the streams to regenerate and  $d_k$  stream  $k$ ’s delay ( $d_k = k - 1$  recovers the single-stem case). Equation (3.7) generalizes to

$$\mathcal{M}(a, b, R) = \{(k, t) : k \in R, a \leq t + d_k < b\}, \quad (3.14)$$

with the loss of (3.8) restricted to  $\mathcal{M}(a, b, R)$ ; a step in  $[a, b]$  whose stream is *not* in  $R$  is fed as an ordinary input, never a target, exactly as  $P, S$  already are. Both properties of Section 3.1.2 survive unchanged: Property 1 concerns  $M^*$  alone, and Property 2’s conditioning set only grows, to include the non-regenerated in-window streams — more conditioning, not a broken factorization. Training composes the two restrictions at random: with probability  $p_{\text{regen}}$ , graded streams are restricted to the chroma-target stem; otherwise every stream in the window is graded. This fine-tune trains from the joint base checkpoint, non-causally, on 10-second segments with batch size 48.

One caveat applies to the loss. Because the model is graded on one stem at a time, the effective supervision per step falls to as few as four of the ten streams, and the per-example normalization of (3.8) rescales the remaining terms so that a target-stem-only step contributes the same total gradient magnitude as a full-window step. Whether that is the right relative weighting between the two, rather than an artifact of reusing one normalizer for both, was not measured.

## Inference

The causal counterpart freezes chosen stems and regenerates the rest against the ordinary streaming KV cache, optionally primed with the training-time coarse-prefix hint — the whole-clip version of “regenerate this instrument.” Editing is extended with an optional stream selection  $R$ : only those streams are marked unknown inside  $[a, b)$ , and the already-per-codebook decode loop needs no other change. This combination is exposed today at the low level and through a standalone test bench, not yet a dedicated high-level method alongside this report’s other editing entry points — a small follow-up (Section 5.3).

This chapter has presented one mechanism and four things built on top of it. The asymmetric span mask makes an interior window sampleable from a causal decoder; the cosine schedule weights the two sides of the context as the window is filled; the prepended chromagram specifies what the window should play; the AdaLN corpus flag specifies what it should sound like; and the stream restriction chooses which instrument it applies to. Each reduces to the identity at its default setting, and each of the two properties the mask is claimed to have was proved above and checked in code. Chapter 4 takes them in the same order and asks which of them survive measurement.

# Chapter 4

## Experiments & Results

This chapter presents the experimental results obtained for the mechanisms introduced in Chapter 3. It first describes the evaluation setup (Section 4.1): the two corpora, the systems under test, the three generation tasks and the metrics used throughout. It then reports text-to-music generation (Section 4.2), evaluated both quantitatively and through a blind listening study. Section 4.3 covers the control layer: melody conditioning, cross-corpus style routing on **schaeffer**, and the conditioner objective of Section 3.2.4. Section 4.4 evaluates editing, on a mixture window and then on the stem axis. The cosine context schedule is not evaluated here and is discussed in Chapter 5.

### 4.1 Setup

#### 4.1.1 Datasets

Every model trains and is evaluated on two corpora of deliberately different character. **shs** is a large, in-house corpus assembled by scraping YouTube, predominantly vocal pop, not the curated, instrumental-only catalogs public MusicGen checkpoints are trained on. Models trained from scratch directly on **shs** face no domain gap to this evaluation corpus, but must learn musical competence from a comparatively small, idiosyncratic corpus with no larger pretraining run to draw on; the public checkpoint is in the opposite position, seeing far more data but of the wrong kind, and every comparison against it should be read with that asymmetry in mind. The corpus is  $\sim 4000$  hours at 32kHz mono with no reliable captions. **schaeffer** is the opposite in every respect. It is the SCHAEFFER corpus [6] — *Spectro-morphological Corpus of Human-annotated Audio with Electroacoustic Features For Experimental Research* — a thousand *sound objects* designed and annotated by composers and students of electroacoustic composition following Pierre Schaeffer’s typo-morphological framework, each carrying a free-text description alongside labels drawn from a closed list of spec-

| Tag              | Content                                                       | Duration      | Format                       | Captions |
|------------------|---------------------------------------------------------------|---------------|------------------------------|----------|
| <b>shs</b>       | scraped YouTube, mostly vocal pop songs                       | $\sim 4000$ h | 32 kHz, mono                 | no       |
| <b>schaeffer</b> | SCHAEFFER: 1000 human-annotated electroacoustic sound objects | $\sim 30$ min | $\leq 44.1$ kHz, $\leq 2$ ch | partial  |

Table 4.1: The two training/evaluation corpora (Section 4.1.1). **schaeffer** is the SCHAEFFER dataset of Berta and Ghisi [6], released CC BY 4.0; it is named for Pierre Schaeffer’s sound-object theory, which its annotation scheme follows, and does not contain his recordings.

tromorphological classes. It is therefore not a corpus of *music* in the sense **shs** is: its items are isolated gestures and textures rather than songs, individually short, roughly 30 minutes in total, up to 44.1 kHz/2 ch, partially captioned in the subset used here, and largely non-tonal — so melody-oriented metrics are cross-checked against chroma cosine rather than trusted alone (Table 4.1).

### 4.1.2 Systems under test

The evaluation is deliberately narrow. **Ours** is a single from-scratch **shs** checkpoint carrying text (CLAP) and melody (chroma) conditioning, fine-tuned with the asymmetric span mask (Section 3.1.2). Every **Ours** row in this chapter is that checkpoint under a different *condition*, not a different model: the chroma is dropped with probability 0.5 during training (Section 3.2.2), so supplying it or nulling it at inference are both in-distribution, and the melody-on/melody-off rows of Tables 4.4 and 4.6 are an ablation of the condition rather than a comparison between two training runs. The same holds for the task axis: generation, continuation and infilling are three ways of calling one decoder.

**Meta** is the public, unmodified MusicGen checkpoint used as an external reference — MusicGen-small for text-to-music, MusicGen-melody for melody conditioning. Stock MusicGen is causal and cannot infill an interior window, so its editing baseline is its *continuation from the prefix*, the closest task it supports; that is a handicap for the baseline, not a fair contest, and Section 4.4.1 reads it accordingly. Two further checkpoints appear once each: the AdaLN **shs+schaeffer** model of Section 3.2.3, which is the system behind both the text-to-music numbers of Section 4.2 and the routing of Section 4.3.2, and the stem model of Section 3.3.1.

### 4.1.3 Tasks and evaluation protocol

Checkpoints are exercised on up to three tasks, mirroring the three-stage pipeline (`generate_eval_set.py`  $\rightarrow$  `compute_metrics.py`  $\rightarrow$  `compare_models.py`): **generation** (full generation from conditioning alone; on a non-causal checkpoint recast as

**generate\_edit** over the whole clip); **continuation** (a fixed-length audio prompt, the rest generated and scored); and **editing/infilling** (an interior window regenerated against frozen two-sided context, undefined for causal checkpoints). For continuation and infilling, metrics are computed on the regenerated region only, both generation and reference cropped to it. The protocol — manifest, sample count, batch size, segment duration, seed, sampling parameters — is fixed identically across every system in a table, so differences across rows are attributable to the model, not to what was asked of it. Audio is cached as loudness-normalized 320 kbps MP3 with per-sample metadata, so scoring is reproducible without re-running the model.

#### 4.1.4 Metrics

Five quantities recur across the tables below, chosen so each table stays compact. **KL divergence (KLD)** runs a pretrained PaSST classifier on generated and reference audio and reports the divergence between the resulting label distributions. Every KLD in this chapter is the single-direction  $\text{KLD}_{pq}$ , from the generated distribution to the reference one, never the symmetrised  $\text{KLD}_{pq} + \text{KLD}_{qp}$  that some pipelines report; the two differ by roughly a factor of two, so the figures here are not comparable with symmetrised ones. It is a broad acoustic-plausibility check that does not require the labels to mean anything musical. **Fréchet Audio Distance (FAD)** fits a Gaussian to each embedding set and compares them,

$$\text{FAD}^2 = \|\mu_x - \mu_y\|^2 + \text{Tr}(\Sigma_x + \Sigma_y - 2(\Sigma_x \Sigma_y)^{1/2}), \quad (4.1)$$

following Kilgour et al. [21], here with a VGGish extractor; its natural *floor*, the same statistic between two disjoint halves of the reference set, is 0.44 on **shs**, so a score is best read as a multiple of it. **CLAP score** compares the CLAP audio embedding of a generated clip against the CLAP text embedding of its description [35], computed on the captioned evaluation subset (the **shs** eval set carries Qwen-generated captions). **Chroma cosine similarity** is the mean frame-wise cosine similarity between generated and reference chromagrams (2.19); the one metric here still well defined on **schaeffer**’s textural material, and the primary measure of melody adherence. **mir\_eval beat F-measure** compares tracked beats under a 70 ms tolerance.

## 4.2 Text-to-music generation

Text-to-music generation is evaluated both quantitatively and through a blind listening study (Table 4.3), on an **shs** evaluation set disjoint from the training set.

The quantitative metrics do not separate the two systems. KLD (1.03 against 1.04) and chroma cosine (0.10 against 0.07) differ by less than a single run at this

| Metric                    | Compares                 | Meaningful on                     | Direction |
|---------------------------|--------------------------|-----------------------------------|-----------|
| KLD <sub>pq</sub> (PaSST) | label distributions      | any                               | lower     |
| FAD (VGGish)              | embedding distributions  | any                               | lower     |
| CLAP score                | audio vs. text embedding | captioned subsets only            | higher    |
| Chroma cosine             | chromagrams              | any (robust on <b>schaeffer</b> ) | higher    |
| mir_eval beat             | beat tracks              | pulsed material                   | higher    |

Table 4.2: Legend for the metrics of Section 4.1.4, reused throughout this chapter.

| System    | Quantitative        |             |             |             |             | Human ( $\sim 20$ ) |                   |
|-----------|---------------------|-------------|-------------|-------------|-------------|---------------------|-------------------|
|           | KLD <sub>pq</sub> ↓ | FAD ↓       | Chroma ↑    | Beat F ↑    | CLAP ↑      | OVL ↑               | REL ↑             |
| Meta      | <b>1.03</b>         | 5.37        | <b>0.10</b> | 0.28        | 0.35        | <b>76.7 ± 3.8</b>   | 74.4 ± 4.4        |
| Ours      | 1.04                | <b>5.04</b> | 0.07        | <b>0.31</b> | <b>0.36</b> | 68.9 ± 4.9          | <b>80.0 ± 4.5</b> |
| Reference | —                   | —           | —           | —           | —           | 76.1 ± 4.4          | 83.7 ± 3.2        |

Table 4.3: Text-to-music generation on **shs**: quantitative metrics and blind human ratings, separated by the vertical rule. Human ratings are overall quality (OVL) and text relevance (REL), on a 1–100 scale, mean  $\pm$  95% CI over 20 raters. The *Reference* row is the ground-truth recording, rated blind alongside the two systems; its quantitative cells are undefined, since it is the target the others are scored against. Best per column in **bold**; FAD is VGGish (floor 0.44).

sample count resolves, and the larger FAD gap (5.37 against 5.04) leaves both far above the reference floor of 0.44 — 12.2 and 11.6 times it respectively, which is the more informative reading of either figure. A 300M model trained from scratch on 4000 hours of **shs** therefore reaches the plausibility band of the public checkpoint.

The listening study separates them along one axis. **Meta** is rated higher on overall quality (76.7 against 68.9) and **Ours** higher on relevance to the prompt (80.0 against 74.4), with the ground-truth reference at 76.1 and 83.7 respectively. The from-scratch model trades audio polish for adherence to the prompt, while remaining below the reference on both axes.

The human study follows MusicGen’s OVL/REL protocol [9]: raters score each clip blind, on overall quality and on relevance to the text prompt, and per-system means are reported with a 95% confidence interval

$$\bar{r} \pm 1.96 \frac{s}{\sqrt{n}}, \quad (4.2)$$

over  $n$  ratings with sample standard deviation  $s$ . The three clips per item (reference, **Meta**, **Ours**) are presented anonymously and all ratings are kept. Ratings are pooled across raters, so the interval reflects variance over items rather than over listeners.

| System                 | $\text{KLD}_{pq} \downarrow$ | $\text{FAD} \downarrow$ | $\text{Chroma} \uparrow$ | $\text{Beat F} \uparrow$ |
|------------------------|------------------------------|-------------------------|--------------------------|--------------------------|
| Ours, chroma supplied  | <b>0.86</b>                  | —                       | 0.65                     | <b>0.32</b>              |
| Ours, chroma nulled    | 1.05                         | —                       | 0.08                     | 0.29                     |
| Meta (MusicGen-melody) | —                            | —                       | <b>0.66</b>              | —                        |

Table 4.4: Melody control on **shs**, **generation** task. Chroma cosine is the primary measure of melody adherence. The first two rows are the *same* checkpoint with the chroma condition supplied and nulled at inference, which the training-time chroma dropout of Section 3.2.2 makes a valid ablation of the condition rather than a comparison between two models. — marks a cell not computed in this run: FAD was not among the metrics requested, and the public MusicGen-melody checkpoint was scored on chroma only, so its row supports a single-column comparison and no more. Best per column in **bold**, over measured cells.

### 4.3 Conditioning and control

The three control paths of Section 3.2 are evaluated in turn: the melody condition (Section 4.3.1), the dataset token routing generation between the two corpora (Section 4.3.2), and the conditioner objective itself (Section 4.3.3). The first two act at inference on a trained checkpoint; the third concerns how the joint embedding carrying the text condition is learned.

#### 4.3.1 Melody control

Supplying the chroma raises melody adherence from 0.08 to 0.65 on the same checkpoint (Table 4.4). Because the two rows differ only in whether the condition is present at inference — the model saw both regimes in training — the eightfold gap is attributable to the conditioning path itself and to nothing else. For scale, the dedicated MusicGen-melody checkpoint reaches 0.66 on the same task, so a condition bolted onto a general checkpoint tracks a target melody about as closely as a model built specifically for it.

Plausibility improves at the same time, KLD falling from 1.05 to 0.86: on free generation the chroma acts as additional information about the target rather than as a competing constraint. The infilling task of Section 4.4.1 shows the opposite sign, the melody there being imposed on a window that must also meet a fixed future.

#### 4.3.2 Cross-corpus style routing on schaeffer

**schaeffer** is a stress test for data scarcity: thirty minutes of annotated electroacoustic sound objects, far too little to train a competent generator from scratch. The AdaLN dataset token (Section 3.2.3) lets one model train on the merged corpus — the  $\sim 4000$ -hour **shs** oversampled against a 10% **schaeffer** fraction — and route generation to either sound world at inference.

Forcing the **schaeffer** token yields  $\text{KLD}_{pq} = 2.33$ , chroma cosine 0.08, beat F-

measure 0.25 and a CLAP score of 0.19 over 512 generations on the **schaeffer** evaluation set; the same checkpoint continuing a prompt from that corpus reaches 1.81, 0.15, 0.37 and 0.16. The chroma and beat figures are low because the material is non-tonal and unpulsed, so they characterise the corpus more than the model, and the KLD is high for the same reason: PaSST’s label distribution is a poor fit for electroacoustic sound objects. With the token forced, the model emits noisy, gestural, unpulsed material of the kind the corpus contains, whereas a thirty-minute corpus on its own supports little beyond a collapsed loop; the effect is audible in the examples on the companion website. A from-scratch **schaeffer**-only model, scored on the same set, would quantify it and was not trained here (Section 5.3).

### 4.3.3 Conditioner objective: multi-label retrieval

The multi-label triplet objective of Section 3.2.4 is scored by cross-modal retrieval on 98 held-out **schaeffer** clips, where chance is  $1/98 \approx 0.01$  at Top-1 (Table 4.5).

Three effects are unambiguous. Frozen LAION-CLAP retrieves at 0.26/0.18, well above chance but far below anything trained on the corpus, so these descriptors are out of distribution for a general-purpose encoder. The triplet term alone collapses the space to 0.05/0.03, the degenerate optimum Section 3.2.4 predicts and the reason it is never used alone. And adapting only the text encoder’s projection head, with its body frozen, gives 0.07/0.06, so the vocabulary has to enter the encoder’s weights rather than be re-aimed at its output.

The comparison the objective was proposed for is not. Against plain contrastive training on the full annotations, the hybrid is identical audio-to-text (0.56) and three points ahead text-to-audio (0.60 against 0.57); across 98 items that gap is not resolvable, and the Top-3 columns that would sharpen it were not computed. The triplet term therefore has no measurable effect at this scale. The larger effect in the table comes from the data rather than the loss: training on the full annotations instead of the descriptors alone is worth eight points at Top-1 and eighteen at Top-3.

## 4.4 Editing

Editing is evaluated on the mixture first (Section 4.4.1), then on the stem axis (Section 4.4.2), the two differing only in whether the regenerated stream set  $R$  of (3.14) is restricted.

### 4.4.1 Infilling a mixture window

Editing is scored as infilling: an interior window is regenerated against frozen two-sided context, and every metric is computed on that window alone (Table 4.6).

| Objective                             | $\lambda_c$ | $\lambda_t$ | Audio $\rightarrow$ text |             | Text $\rightarrow$ audio |             |
|---------------------------------------|-------------|-------------|--------------------------|-------------|--------------------------|-------------|
|                                       |             |             | Top-1                    | Top-3       | Top-1                    | Top-3       |
| LAION-CLAP (frozen, zero-shot)        | —           | —           | 0.26                     | —           | 0.18                     | —           |
| Contrastive, descriptors only         | 1.0         | 0.0         | 0.48                     | 0.60        | 0.47                     | 0.58        |
| Contrastive, full annotations         | 1.0         | 0.0         | <b>0.56</b>              | <b>0.78</b> | 0.57                     | <b>0.79</b> |
| Multi-label triplet only              | 0.0         | 1.0         | 0.05                     | —           | 0.03                     | —           |
| Contrastive + multi-label triplet     | 1.0         | 0.1         | <b>0.56</b>              | —           | <b>0.60</b>              | —           |
| Text encoder frozen (projection only) | 1.0         | 0.0         | 0.07                     | —           | 0.06                     | —           |

Table 4.5: Cross-modal retrieval on 98 held-out **schaeffer** clips, for the conditioner objectives of Section 3.2.4; chance is 0.01 (Top-1) and 0.03 (Top-3).  $\lambda_c$  and  $\lambda_t$  are the InfoNCE and triplet weights of (3.12). Best per column in **bold**; — marks a cell not computed. The Top-3 columns are missing for four of the six rows, so the proposed objective and the contrastive baseline can only be compared at Top-1, which is part of why this branch is reported as inconclusive rather than as a result.

| System (task)                     | KLD <sub>pq</sub> ↓ | Chroma ↑    | Beat F ↑    | CLAP ↑ |
|-----------------------------------|---------------------|-------------|-------------|--------|
| Meta — continuation (prefix only) | 0.81                | 0.16        | 0.31        | —      |
| Ours — infill, no melody          | <b>0.61</b>         | 0.14        | 0.31        | —      |
| Ours — infill, melody             | 0.72                | <b>0.42</b> | <b>0.39</b> | —      |

Table 4.6: Editing as infilling on **shs**, metrics on the regenerated window only. Meta cannot infill an interior window, so its row is a prefix-only continuation, the closest baseline; it is scored over 300 samples against 64 for the two **Ours** rows, so the bold marks the better of the two comparable rows rather than a three-way winner. Best per column in **bold**. The CLAP column is — throughout because **shs** carries no captions, so no sample contributes to the text-consistency score on this evaluation set; measuring it requires the machine-captioned variant used in Table 4.3.

The baseline row is a prefix-only continuation, which writes into the same window without being told what follows it. What the table does support is the effect of the melody condition, both rows coming from the same checkpoint and the same run. Supplying the melody triples chroma agreement inside the window,  $0.14 \rightarrow 0.42$ : unlike free generation (Section 4.3.1), the condition here trades against plausibility, since the window must satisfy the melody and land on a fixed future simultaneously. Audio examples of these edits are on the companion website.<sup>1</sup>

#### 4.4.2 Stem-by-time editing

The stem model — both plain multi-stem generation and the stem $\times$ time editor built on top of it (Section 3.3.1) — is not scored quantitatively in this report. The checkpoint was still at epoch 120 of a 2000-epoch schedule (240k steps at batch size 48) when the internship’s compute budget ran out: the training and validation curves were still descending, well short of the convergence a comparison against the other systems in

<sup>1</sup>Companion website with audio examples: <https://emilio-pcrd.github.io/editgen-demo/>.

this chapter would need. Reaching that point, and running the evaluation pipeline used throughout this chapter on top of it, was a question of remaining time rather than of anything left to design; the protocol is specified below precisely so that running it is the only step left.

Preliminary, unscored experiments run locally against this early checkpoint — decoding a chosen stem inside a chosen window and listening to the result — already show the mechanism behaving as intended: the frozen stems come back audibly untouched and the regenerated one lands inside the window, the same qualitative signature Section 4.4.1 measures numerically for the mixture case. This is evidence that the composition of the mask with the stream restriction ((3.14)) works end to end, short of the quantitative confirmation a converged checkpoint would give.

The evaluation protocol is defined as follows: decode to the four separated stems rather than the summed mix, and report  $\text{KLD}_{pq}$  and chroma cosine on the target stem alone for (a) whole-clip stem regeneration, freezing three stems and regenerating the fourth; (b) the stem $\times$ time edit, with  $R$  restricted to one stem’s streams inside an interior window; and (c) the same checkpoint with  $R$  unrestricted. Running it on **other** and on **bass** separately would also test the loss weight of (3.13), since an under-trained bass stream showed up as a weak link in the early experiments, likely because of the data, where a lot of data does not have a bass, making the model to learn to zero it out. This still decrease the loss during training, wreaching a weak local minimum that we want to avoid.

Qualitatively, stem generation was the setting most sensitive to training budget, and the runs were still improving when they were stopped; whether this reflects optimisation budget or model capacity is what the protocol above would determine.

# Chapter 5

## Discussion and Conclusion

The question driving this work was whether an autoregressive music model can be made revisable without being rebuilt. Existing editors reach infilling by leaving the autoregressive paradigm, by reordering the token sequence, or by iterative refinement, and each pays for it somewhere else. The hypothesis tested here is narrower: that infilling is a property of *attention visibility* rather than of architecture, and that changing which positions may see which suffices to turn a generator into an editor while keeping sampling, guidance and the key-value cache intact.

### 5.1 Contributions and evidence

Revising a passage means sampling  $p(E \mid P, S)$  (3.1): regenerating an interior window so that it follows its prefix *and* lands on its suffix. A causal decoder cannot, being blind to the future it has to meet.

This work proposes an asymmetric span mask  $M^*$  (Section 3.1.2), which lets the edit window attend to the whole prefix and suffix while every position outside it keeps an ordinary view. Two properties follow: the context’s hidden states, hence its key-value cache, are independent of whatever the window generates (Property 1), and generation inside the window stays strictly left-to-right (Property 2). Together they confine the change to the mask — sampling, truncation, classifier-free guidance and the decoding loop carry over untouched, and the causal model is recovered exactly as a degenerate case. An edit-mode training recipe (Section 3.1.3) turns that into a working editor, and three switchable control layers (a cosine context schedule, a chroma path, an AdaLN dataset token) make the edit directable, each reducing to the unmodified model at its default setting. Because a stem-tokenized clip is more codebooks rather than a different architecture, the same mask composed with a stream selection (Section 3.3.1) edits one instrument inside one window on top of MusicGen-Stem [29].

Chapter 4 supports three claims. Melody conditioning works, and the effect is cleanly attributed: supplying the chroma to one checkpoint raises adherence from 0.08

to 0.65, against 0.66 for a checkpoint built for melody alone (Section 4.3.1). Infilling behaves as designed: imposing a melody on a window that must also meet a fixed future trades legibly against plausibility — chroma adherence rising  $0.14 \rightarrow 0.42$  for a cost of 0.11 in KLD,  $0.61 \rightarrow 0.72$  (Section 4.4.1). And a 300M model trained from scratch on 4000 hours reaches the plausibility band of the public checkpoint, listeners placing it lower on polish and higher on prompt adherence (Section 4.2).

## 5.2 Discussion

### 5.2.1 Positioning against the alternative editing families

Table 2.1 mapped four families by what each gives up. Against masked iterative decoding [8, 15, 37], Property 2 gives an *exact* factorization with no independence assumption between tokens filled in the same round. That advantage is currently theoretical: the decode recomputes a full forward pass per generated frame, because the streaming cache cannot yet be reused across an edit (Section 3.1.3), so an uncached edit is slower today than an  $R$ -round refinement — the reverse of the architectural argument. Property 1 makes this recoverable rather than intrinsic, licensing  $K_P, V_P, K_S, V_S$  once and  $O(|E|)$  thereafter, which is why the cached decode heads Section 5.3.

Latent-diffusion inpainting [13] has no direct analogue of the per-token controls — temperature, top- $k/p$ , CFG — that a composer’s retry workflow leans on; staying autoregressive was a choice to keep that toolbox, not a claim about diffusion. Fill-in-the-middle training [4, 12, 14] also keeps a causal decoder and, for text, costs essentially nothing [4]. It is not free here: audio tokens sit on a lattice the delay pattern staggers, so reordering breaks the alignment between codebooks, and changing visibility instead cost a new mask, delay-aware bookkeeping in loss and decode, and a dedicated recipe — a price the symbolic-music precedent [18] does not pay. Finally, a longer context trained outright [11] buys single-pass coherence at the cost of a multi-stage hierarchy.

### 5.2.2 The small-compute regime and its limits

Every configuration fits on  $2 \times 48$  GB GPUs, every corpus is small or mismatched, and every mechanism was validated on its own checkpoint. This was the premise (Section 1.1): a technique that only works at industrial scale does not reach the labs and independent artists it was meant for. In practice two workstation GPUs reached good generative quality in a week of training, and clearly stronger quality in two. It bought reproducibility, and evidence that the mechanisms are *architectural* rather than scale-dependent — the stem editor has the largest apparent scope of anything here and adds nothing to the shared budget, more stems costing parameters rather than a new order of compute.

It cost more. First, a quality ceiling: every system but the public reference trains from scratch on a budget measured in days, visible in Section 4.2 where listeners rank the public checkpoint higher on overall quality despite its domain mismatch. The stem model is most starved by this, and whether its gap is optimisation budget or model capacity is what Section 4.4.2’s protocol would decide. Second, no single checkpoint carries the span mask, the melody path, the AdaLN token and the stem extension at once (Section 3.2.3); composability is argued architecturally, not shown, and the AdaLN oversampling, the weak chroma bottleneck and training from scratch are all responses to scarcity that might not survive at scale. Third, data: **shs** has no human captions, its prompts coming from a Qwen captioner, so text adherence is learned against an approximate signal and the relevance axis of Section 4.2 measures agreement with that annotator rather than a human one. Two engineering caveats remain: the decode is not cache-accelerated, which rules out interactive use, and the stem models inherit an uncorrected ceiling from Hybrid Transformer Demucs [27], most consequential on **schaeffner**.

### 5.3 Future work and perspectives

We already investigated some direct improvements of this work for further research.

**Changes of scale or recipe.** One checkpoint carrying every mechanism is the largest gap the budget could not close, and the one that would turn the architectural argument for composability into a demonstration. For long context, training with sliding-window attention and a non-causal prefix — so inference iterates from a bootstrap sample that stays on-distribution across a piece — is a cleaner answer than stitching windows, which accumulates degeneration seam by seam. The idea came from [22]. In addition, a codec at 20 Hz or below, as Lyria [16] and ACE-Step [1] adopt. It could lead to earlier convergence of the stem model, at now it needs to train for many more epochs. Most speculative is reinforcement learning from the model’s own samples: sampling repeatedly from one prompt, keeping the best draw by human rating or an automatic proxy such as FAD, and fine-tuning towards it.

**Toward composer-facing tools.** The motivation was never audio quality for its own sake, but an instrument a composer can retry against — local, reversible, addressable track by track. A local test bench (“Stem Studio”) already puts the models behind a DAW-like timeline: four stem tracks, drag-to-select a window, generate or infill it.

Whether a generative model of music can be composed with rather than only queried is not answered here. What this work establishes is narrower: local regeneration and

musical control, applied instrument by instrument, require neither leaving the autoregressive paradigm, nor an architecture beyond one reshaped attention mask and the recipe that trains it, nor more compute than two workstation GPUs. Whether the audio they produce is good enough to change how a piece gets written is what the next round of experiments has to answer. Last enhancement would be to run this on a larger model using more different stems, using a dataset with more stems, in order to have many more diversity and control over the instrument generated.

# Bibliography

- [1] ACE Studio and StepFun. ACE-Step: A step towards music generation foundation model, 2025. <https://github.com/ace-step/ACE-Step>.
- [2] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzett, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi, M. Sharifi, N. Zeghidour, and C. Frank. Musiclm: Generating music from text. *arXiv preprint arXiv:2301.11325*, 2023.
- [3] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [4] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.
- [5] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- [6] M. Berta and D. Ghisi. SCHAEFFER: A dataset of human-annotated sound objects for machine learning applications. In *Proceedings of the 28th International Conference on Digital Audio Effects (DAFx)*, 2025. <https://github.com/mauriziobrt/SCHAEFFER>.
- [7] Z. Borsos, R. Marinier, D. Vincent, E. Kharitonov, O. Pietquin, M. Sharifi, D. Roblek, O. Teboul, D. Grangier, M. Tagliasacchi, and N. Zeghidour. Audioldm: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.
- [8] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman. Maskgit: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [9] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez. Simple and controllable music generation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research (TMLR)*, 2023.
- [11] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever. Jukebox: A generative model for music. *arXiv preprint arXiv:2005.00341*, 2020.
- [12] C. Donahue, M. Lee, and P. Liang. Enabling language models to fill in the blanks. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [13] Z. Evans, C. Carr, J. Taylor, S. H. Hawley, and J. Pons. Fast timing-conditioned latent audio diffusion. In *International Conference on Machine Learning (ICML)*, 2024.
- [14] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis. Incoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations (ICLR)*, 2023.
- [15] H. F. Garcia, P. Seetharaman, R. Kumar, and B. Pardo. Vampnet: Music generation via masked acoustic token modeling. In *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023.
- [16] Google DeepMind. Lyria: Google deepmind’s music generation model, 2023. <https://deepmind.google/technologies/lyria/>.
- [17] R. M. Gray. Vector quantization. *IEEE ASSP Magazine*, 1(2):4–29, 1984.
- [18] G. Hadjeres and L. Crestel. The piano inpainting application. *arXiv preprint arXiv:2107.05944*, 2021.
- [19] J. Ho and T. Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.
- [20] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.
- [21] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi. Fréchet audio distance: A reference-free metric for evaluating music enhancement algorithms. In *Proceedings of Interspeech*, 2019.
- [22] T. Lemerle, N. Obin, and A. Roebel. Small-e: Small language model with linear attention for efficient speech synthesis,

2024. URL  
<https://arxiv.org/abs/2406.04467>.
- [23] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
  - [24] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
  - [25] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2018.
  - [26] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21, 2020.
  - [27] S. Rouard, F. Massa, and A. Défossez. Hybrid transformers for music source separation. *arXiv preprint arXiv:2211.08553*, 2022.
  - [28] S. Rouard, Y. Adi, J. Copet, A. Roebel, and A. Défossez. Audio conditioning for music generation via discrete bottleneck features. In *International Society for Music Information Retrieval Conference (ISMIR)*, 2024.
  - [29] S. Rouard, R. San Roman, Y. Adi, and A. Roebel. MusicGen-Stem: Multi-stem music generation and edition through autoregressive modeling. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.
  - [30] F. Schroff, D. Kalenichenko, and J. Philbin. FaceNet: A unified embedding for face recognition and clustering. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
  - [31] N. Tokui. *Tsukuru tame no AI: Kikai to sōzōsei no hateshinai monogatari [AI for Creating: The Never-Ending Story of Machines and Creativity]*. BNN, Tokyo, 2021. In Japanese.
  - [32] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.
  - [33] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
  - [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
  - [35] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov. Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.
  - [36] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021.
  - [37] A. Ziv, I. Gat, G. L. Lan, T. Remez, F. Kreuk, A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. Masked audio generation using a single non-autoregressive transformer. In *International Conference on Learning Representations (ICLR)*, 2024.